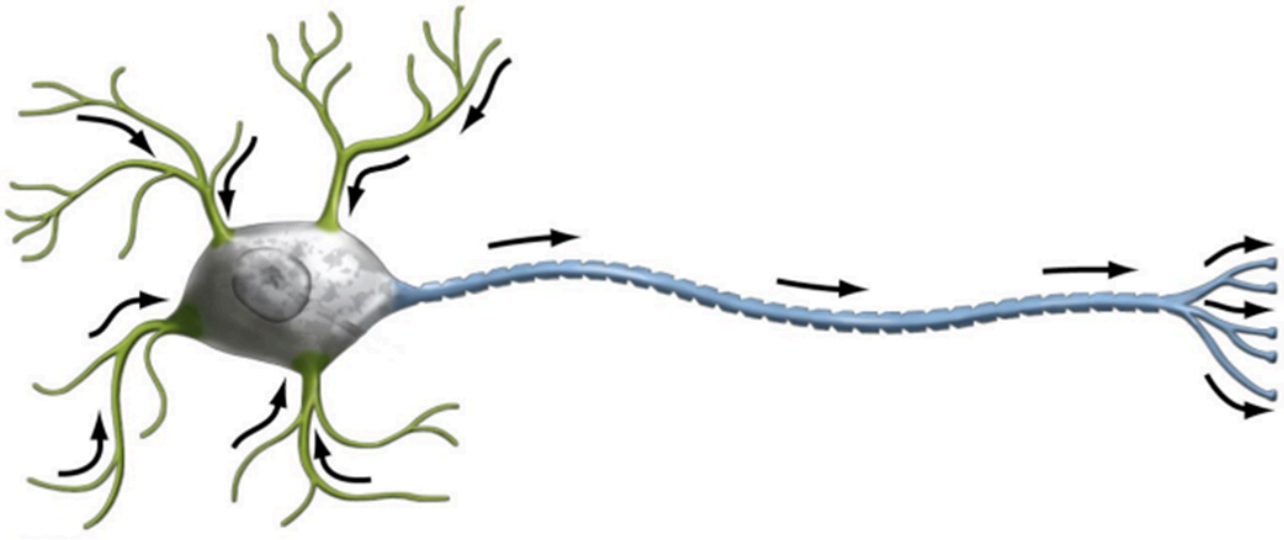
UNIDAD 2

Perceptrón Simple

La inspiración biológica en redes neuronales: fisiología neuronal básica, redes de neuronas biológicas y escalas de organización estructural del cerebro.

Neurona biológica



Dendritas: conforman un árbol dendrítico y permite que la neurona reciba señales. Cada señal puede ser excitatoria (+) que ayuda a que la neurona se active, o Inhibitoria (-) que evita que la neurona se active y de distintas intensidades.

Núcleo (contenido en el soma – cuerpo neuronal): donde se procesa la información. La membrana del núcleo tiene una polaridad, y cuando la carga que se acumula en esta membrana supera un umbral, la neurona se despolariza y manda un único pulso por el axón. Si no supera el umbral, se encuentra en reposo, esto se conoce como comportamiento todo/nada.

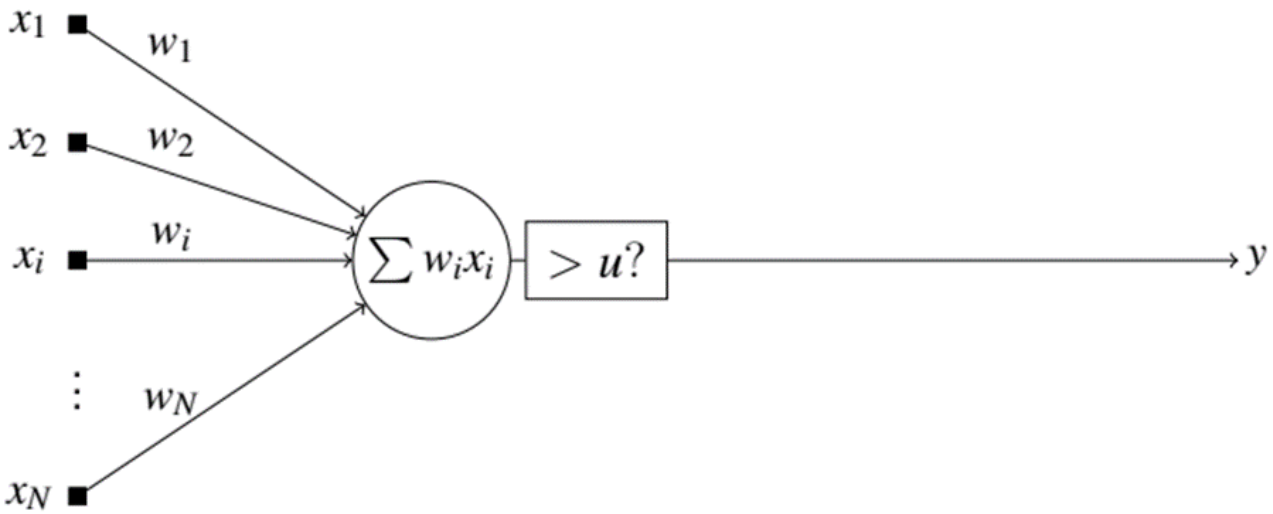
Axón: por donde la neurona da su salida.

Sinapsis es el proceso de transmisión de los neurotransmisores.

- La corteza cerebral . . . 10^11 neuronas
- Redes de neuronas
    - **No linealidad:** la combinación de dos entradas no es una simple suma, y que un pequeño cambio en la entrada puede generar un cambio grande en la salida.
    - **Paralelismo:** hay muchas neuronas que, para resolver problemas complejos, se dividen en unidades de procesos.

- **Capacidad de Aprendizaje:** aprenden a partir de datos o ejemplos
- **Generalización:** capacidad de resolver problemas nunca antes vistos.
- **Adaptabilidad:** capaces de adaptarse a nuevas situaciones
- **Robustez:** capacidad de funcionar bien incluso cuando faltan datos o existe ruido.

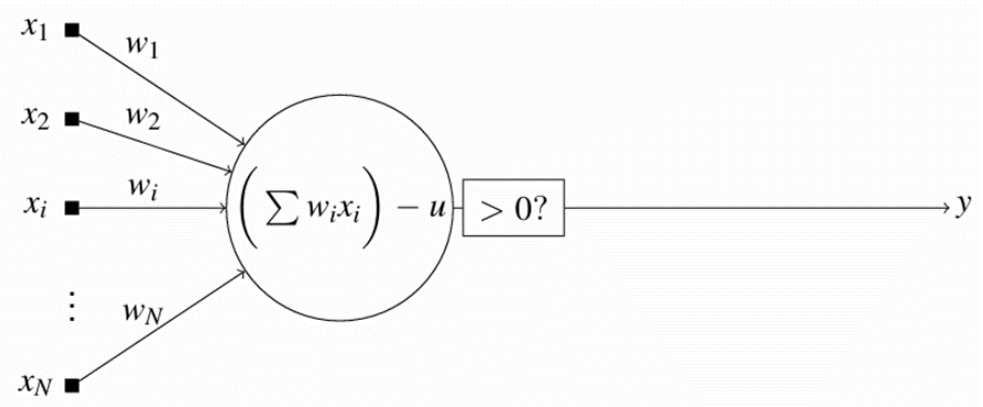
MODELOS DE NEURONA



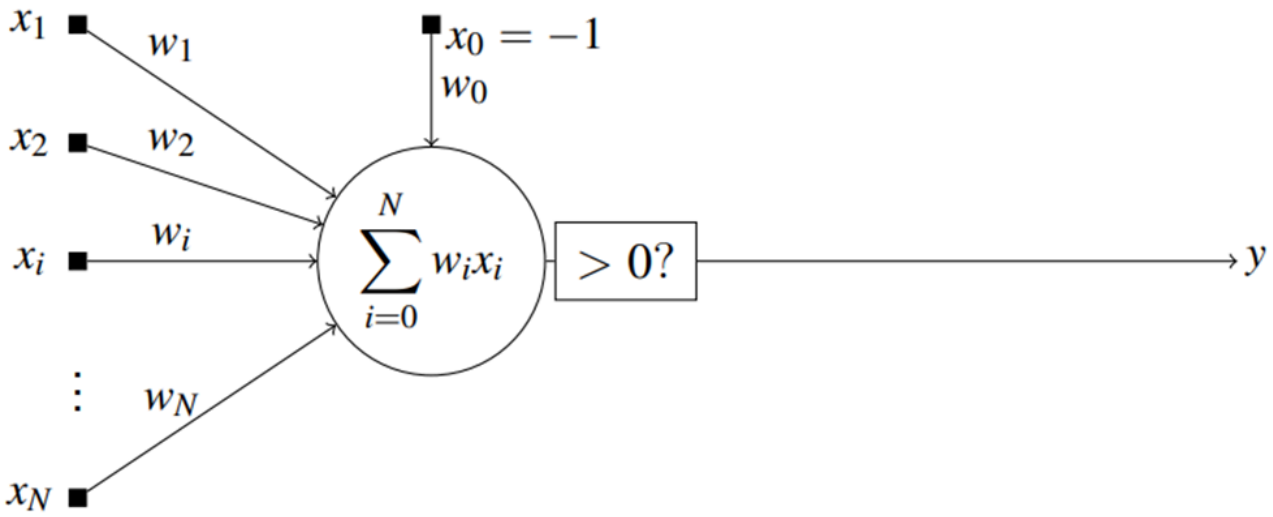
- Tiene x_N entradas y cada entrada x_I tiene un peso sináptico w_I , si la sinapsis es excitatoria será positivo y si es inhibitoria será negativo. De esta manera se ponderan las entradas de acuerdo a la importancia que tenga para la despolarización de la neurona.
- u es el umbral que debe superar la sumatoria para que la neurona de una salida , la cual puede ser + y -1 o +1 y 0.
- Se supone que todos los estímulos llegan simultáneamente.

Se hace una suma ponderada y se verifica si supera el umbral. Si lo supera da la salida .

Simplificación restando el umbral



Simplificando más



Para simplificar aca agregamos una entrada $x0 = -1\, y\, w0 = u$ y cambiamos el índice de la sumatoria que antes iba de i=1 y ahora de i=0

Modelo matemático

- Modelo matemático del perceptrón simple
  - Producto interno y umbral: $y = \phi(v - u) = \phi\left(\sum_{i=1}^{N} w_i x_i - u\right)$
  - Entrada extendida: $x_0 = -1, w_0 = u$

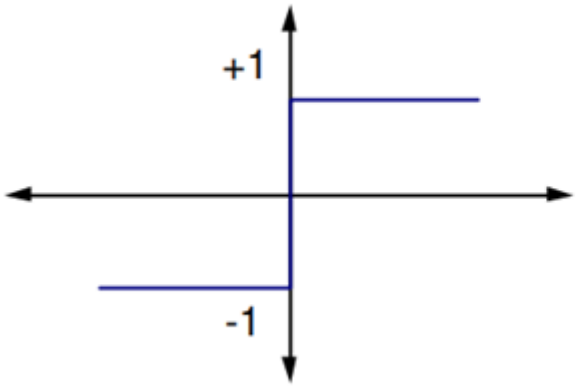
$$y = \phi\left(\sum_{i=0}^{N} w_i x_i\right) = \phi\left(\langle \mathbf{w}, \mathbf{x} \rangle\right)$$

v se llama activación lineal, es el umbral. Nosotros usamos la entrada extendida, que se puede ver como un producto interno o escalar entre el vector de pesos sinápticos y el vector de entradas, esto da un escalar que indica el nivel de excitación de la neurona.

Funciones de activación ϕ

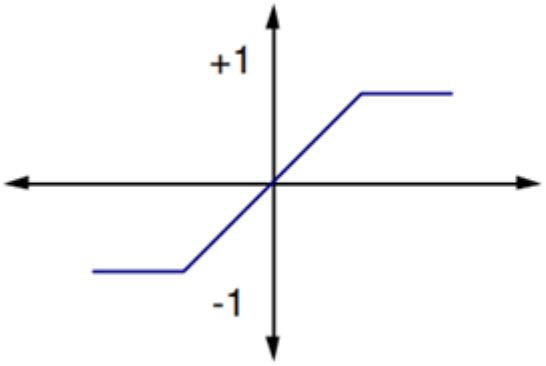
Sirven para verificar si se activa o no la neurona.

- Funciones de activación $\phi(z)$



$$\text{sgn}(z) = \begin{cases} -1 & \text{si } z < 0 \\ +1 & \text{si } z \geq 0 \end{cases}$$

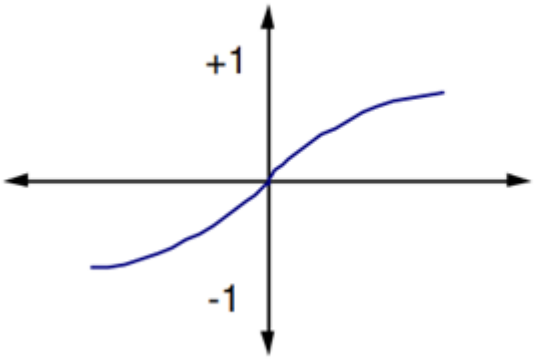
- Funciones de activación $\phi(z)$



$$\text{sln}(z) = \begin{cases} -1 & \text{si } z < -a \\ \alpha z & \text{si } -a < z < 0 \\ +1 & \text{si } z \geq a \end{cases}$$

Presenta una linealización de la zona de transición, se evita la discontinuidad, cambia la pendiente de la función.

- Funciones de activación $\phi(z)$

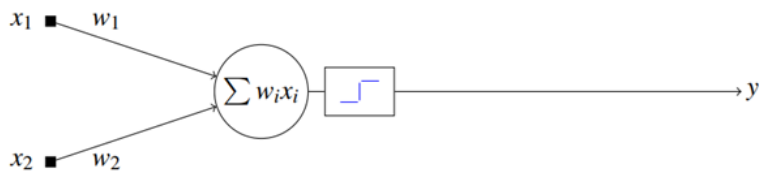


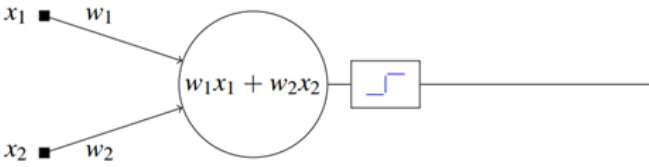
$$\text{sig}(z) = \frac{1 - e^{-az}}{1 + e^{-az}}$$

SIGMOIDE

Función sigmoidea. Es no lineal, no presenta discontinuidad. La constante cambia la pendiente de la función, entre -1 y +1. Cuanto más grande el , mas parecida al signo. Sigue pudiendo modelar el comportamiento todo o nada, pero ahora es derivable.

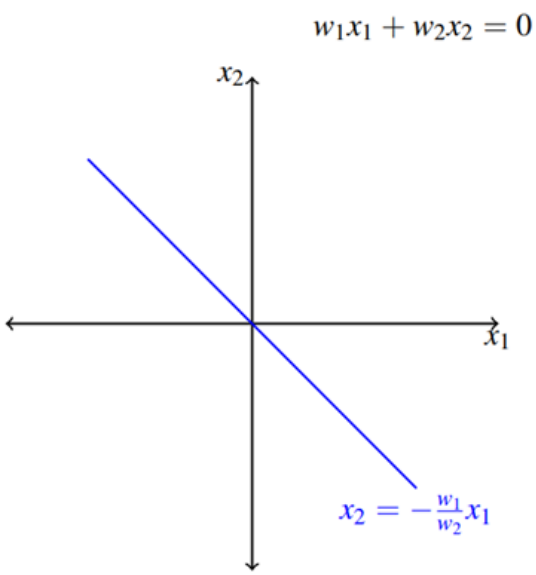
Un perceptrón simple con 2 entradas, y fda signo:

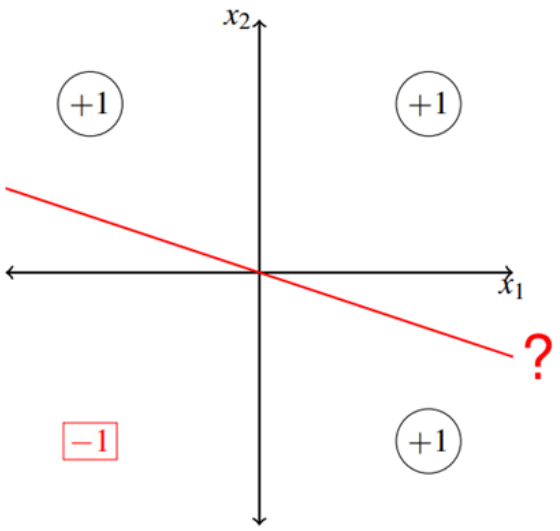




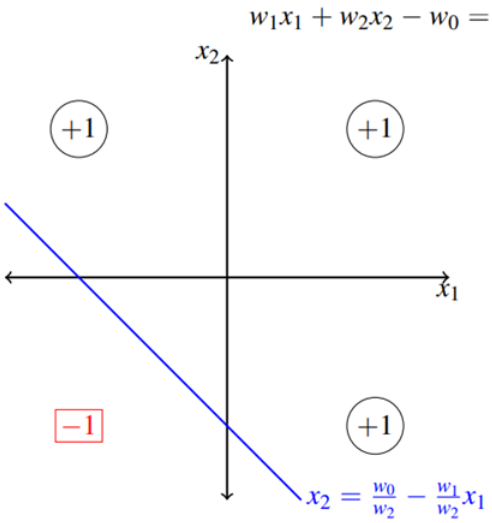
$$y = \text{sgn}(w_1x_1 + w_2x_2)$$

Caso del OR





no me permite plantear el or, para eso necesitamos el sesgo/bias que es la entrada x0=-1 y w0=u la que nos permite mover la recta



Entrenamiento: Algoritmos de aprendizaje

Existen dos métodos de entrenamiento para el perceptrón simple, **el método de corrección de error** donde no importa que funcion de activacion usemos y el **método del gradiente** donde la función de activación debe ser derivable ya que necesitamos buscar un gradiente

CORRECCIÓN DE ERROR

El objetivo es mostrarle ejemplos de los patrones de entradas con las salidas deseadas y que solo encuentre el valor de los pesos sinápticos

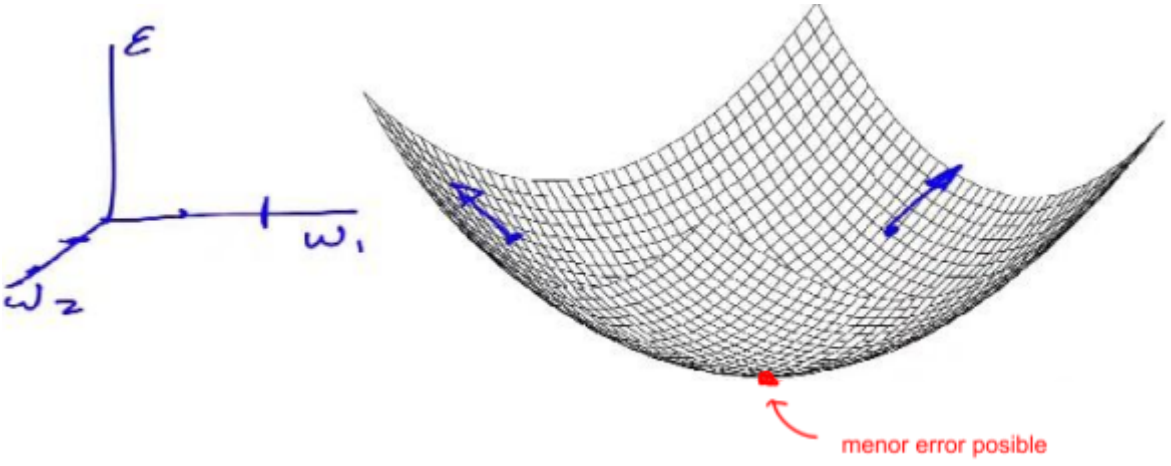
1. Inicializar al azar los pesos w [-0.5 , 0.5]
2. Se muestran muchos ejemplos con las salidas esperadas y en cada caso:
   a. SI la salida de la red es correcta no se hacen cambios. PRINCIPIO DE MÍNIMA PERTURBACIÓN
   b. SI la salida de la red es incorrecta actualizamos los pesos w en el sentido opuesto con el que contribuyeron a la salida incorrecta usando la fórmula
3. Volver a 2 hasta alcanzar un criterio de finalización

FÓRMULA DE ACTUALIZACIÓN

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \frac{\eta}{2}[y_d(n) - y(n)]\mathbf{x}(n)$$

ENTRENAMIENTO POR MÉTODOS DE GRADIENTE

El concepto es mover los pesos en la dirección en que se reduce el error, dirección que es opuesta a su gradiente con respecto a los pesos
El gradiente apunta en la dirección en donde la curva o superficie del error crece, vamos a movernos iterativamente en la dirección opuesta



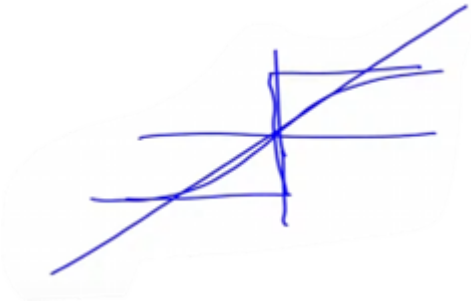
Ecuación básica:

$$\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_w \xi(\mathbf{w}(n))$$

pesos en la it. sig. = peso actual - vel. de aprendizaje*gradiente de error respecto a los pesos

PERCEPTRÓN SIMPLE: MÉTODO DEL GRADIENTE CASO LINEAL

¡¡¡Eliminamos la función de activación para simplificar, usamos simplemente una recta!!!



Criterio del error instantáneo:

$$e^2(n) = [d(n) - y(n)]^2 = [d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]^2$$

d es la salida deseada

declaramos el error (al cuadrado para que siempre sea positivo y se vaya acumulando

Aplicamos el gradiente del error respecto a los pesos, es decir la derivada del error respecto a los pesos. d(n) es una constante que está sumando y x(n) es una constante que esta multiplicando, por lo tanto solo queda -x(n)

Bajo el dos de exponente, multiplicó por lo que ya tengo

$$\nabla_w e^2(n) = 2 \underbrace{[d(n) - \langle \mathbf{w}(n), \mathbf{x}(n) \rangle]}_{\text{error e(n)}} (-\mathbf{x}(n))$$

$$\nabla_w e^2(n) = 2e(n)(-\mathbf{x}(n))$$

reemplazando en: $\mathbf{w}(n+1) = \mathbf{w}(n) - \mu \nabla_w \xi(\mathbf{w}(n))$

$$\boxed{\mathbf{w}(n+1) = \mathbf{w}(n) + 2\mu e(n)\mathbf{x}(n)}$$

CLAVE SABER ESTO REEMPLAZANDO PARA LLEGAR A LA FÓRMULA

desarrollar paso a paso mejor para entenderlo

Primero definimos un criterio de error $\xi = e^2(n)$

$e^2(n) = [d(n) - y(n)]^2$ $\quad y(n) = \langle w(n), x(n) \rangle$ reemplazamos y(n)

$e^2(n) = [d(n) - \langle w(n), x(n)\rangle]^2$ aplicamos gradiente ahora

$\nabla_w e^2(n) = \nabla_w [d(n) - \langle w(n), x(n)\rangle]^2$ ahora derivamos respecto a w

tomamos $u = d(n) - \langle w(n), x(n)\rangle \nabla u = - x(n)$

$\nabla_w e^2(n) = d[u]^2 * \nabla u$

$\nabla_w e^2(n) = 2[u] * \nabla u$

$\nabla_w e^2(n) = 2[d(n) - \langle w(n), x(n)\rangle] * - x(n)$ $d(n) - \langle w(n), x(n)\rangle ES\ EL\ ERROR$

$\nabla_w e^2(n) = 2[e(n)] * - x(n)$

ECUACIÓN BÁSICA

$\xi(w(n)) = error = (yd - y)$

lo elevamos al cuadrado para que sea siempre positivo y se vaya acumulando

$\xi(w(n)) = error^2 = (yd - y)^2$

$w(n+1) = w(n) - u\nabla_w \xi(w(n))$ reemplazamos lo que obtuvimos

$w(n+1) = w(n) + 2u[e(n)] * x(n)$

QUE PASA SI USAMOS SIGMOIDE EN EL MÉTODO DE GRADIENTE PARA PERCEPTRÓN SIMPLE?

$y(n) = \phi(v)$ $v(n) = \langle w(n), x(n)\rangle$

$e^2(n) = [d(n) - y(n)]^2$ -> $e^2(n) = [d(n) - \phi(v)]^2$ ->

$e^2(n) = [d(n) - \phi(\langle w(n), x(n)\rangle)]^2$

aplicamos gradiente en ambos lados

$\nabla_w e^2(n) = \nabla_w [d(n) - \phi\langle w(n), x(n)\rangle]^2$

tomamos $u = d(n) - \phi\langle w(n), x(n)\rangle$

$\nabla_w e^2(n) = \nabla_w [u]^2$

$d(u^2)/du = 2u$

$\nabla_w e^2(n) = 2u * \nabla_w [u]$

$\nabla_w e^2(n) = 2u * \nabla_w [d(n) - \phi\langle w(n), x(n)\rangle]$distribuimos el gradiente

$\nabla_w e^2(n) = 2u * [\nabla_w d(n) - \nabla_w \phi(\langle w(n), x(n)\rangle)]$ d(n) no tiene w así que es 0

$\nabla_w e^2(n) = 2u * - \phi'(v(n)) * x(n)$

$\nabla_w e^2(n) = 2(d(n) - \phi\langle w(n), x(n)\rangle) * - \phi'(v(n)) * x(n)$

$$\nabla_w e^2(n) = 2(e) * \quad - \phi'(v(n)) * x(n)$$

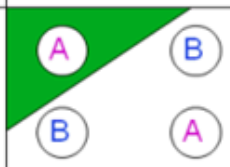
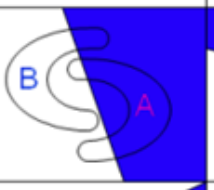
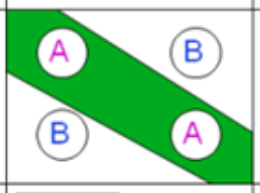
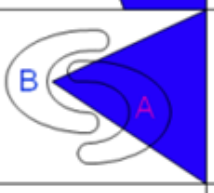
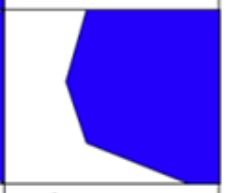
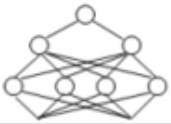
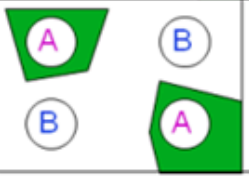
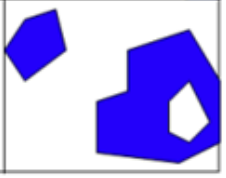
$$w(n+1) = w(n) + 2u(e) * \phi'(v(n)) * x(n)$$

UNIDAD 3

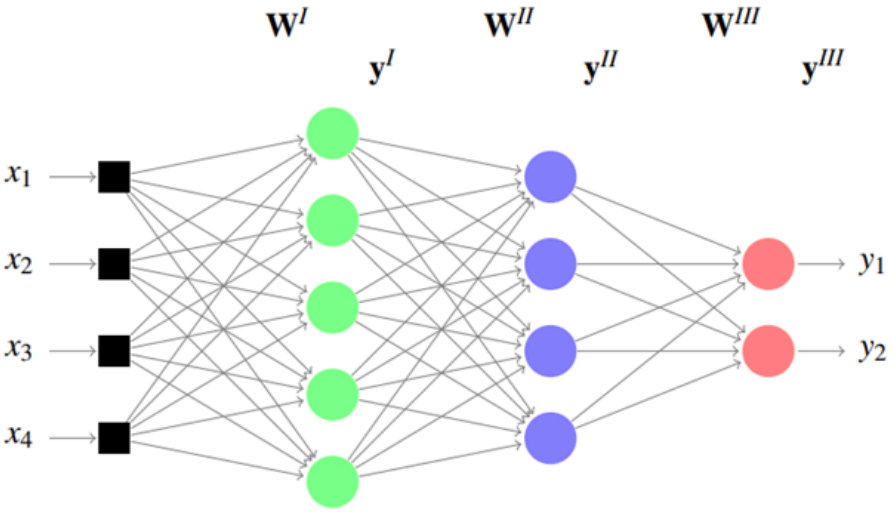
Perceptrón Multicapa (MLP)

El perceptrón multicapa puede resolver problemas más complejos que un perceptrón simple, como por ejemplo el problema del XOR.

Regiones de decisión

Estructura	Tipos de regiones de decisión	Problema XOR	Separación en clases	Formas regiones más generales
Una capa 	hemiplano limitado por hiperplano			
Dos capas 	Regiones convexas abiertas o cerradas			
Tres capas 	Arbitrarias (Complejidad limitada por N°. de Nodos)			

Notación



La matriz W^I tendrá todos los pesos entre las entradas y la primer capa, es decir va a tener 5*4=20 pesos

El vector de salidas de la primer capa y^I tendrá 5 salidas

La matriz W^{II} tendrá todos los pesos entre las salidas de y la segunda capa, es decir va a tener 4*5=20 pesos

El vector de salidas de la segunda capa y^{II} tendrá 4 salidas

La matriz W^{III} tendrá todos los pesos entre las salidas de y la tercer capa, es decir va a tener 2*4=8 pesos

El vector de salidas de la tercer capa y^{III} tendrá 2 salidas, las salidas finales

1 Cálculo de las salidas en cada capa

- Capa 1: $y^I = \phi(W^I x) = \phi(v^I)$

$$v^I = \sum_{i=0}^{N} w^I_{ji} x_i$$

j barre la neurona de la capa actual es decir j=1 neurona 1 j=2 neurona 2

i barre la neurona o entrada de la CAPA ANTERIOR

w_12 = peso de la neurona 2 de la capa anterior que contribuye a la neurona de salida 1

- Capa 2: $y^I = \phi(W^I x) = \phi(v^I)$
- Capa 3: $y^I = \phi(W^I x) = \phi(v^I)$

La función de activación que se usa es

$\phi(z) = \frac{2}{1+e^{-bz}} - 1$ que es simétrica ± 1.

el exponente b es para controlar que tan brusca es la transición de -1 a +1

z sería lo que queremos "comprimir" dentro de la sigmoide en este caso v=(x(n)*w(n))

Se usa esa función de activación ya que para el método del gradiente necesito poder derivar

Filas de **W**: neuronas de capa siguiente, columnas, neuronas de capa anterior.

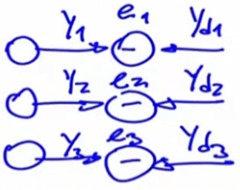
2 Propagación hacia atrás

Criterio de error

Comenzamos definiendo un criterio de error

Para cada neurona habrá un error al instante o patrón . Para el error total, la función de error que usaremos es la Suma del error cuadrático instantáneo:

$$\xi(n) = \frac{1}{2}\sum_{j=1}^{M} e_j^2(n)$$



Se genera un vector de errores comparando el valor deseado con el valor obtenido. El error es para la iteración n.

- Se eleva al cuadrado ya que si el error de una neurona es -1 y la de otra es +1, el error sería 0 lo cual no es correcto.
- El ½ está por conveniencia ya que más adelante se derivará bajando el 2.

Aplicación del gradiente (caso general)

Como en el caso de un perceptrón simple, se modificarán los pesos en el sentido opuesto al que hacen crecer al gradiente error. Nuevamente, u es la velocidad de aprendizaje, que tan rápido se ajustan los pesos. Si modifico mucho los pesos, pierde o se olvida lo que aprendió en los ejemplos anteriores. Si la velocidad es baja va a tardar muchísimo.

Lo que debo sumar a los pesos será:

$$\Delta w_{ji}(n) = -\, u \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

Con regla de la cadena,

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

La derivada del criterio de error respecto a los pesos es:

- El criterio de error cuadrático medio depende del error cuadrático de la neurona j que nos interesa
- A su vez el error cuadrático de esa neurona j depende de la salida que haya dado esa neurona
- A su vez la salida de la neurona depende de la salida lineal
- Y la salida lineal es lo que depende de forma directa de los pesos

$e = yd - y \;\; -> \; y = \phi(v) \;\; -> \;\; v = \{x(n), w(n)\}$

$v_j(n) = \sum_{i=0}^{N} w_{ji}(n) y_i(n)$ *entonces*:

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = \frac{\partial \sum_{i=0}^{N} w_{ji}(n) y_i(n)}{\partial w_{ji}(n)}$$ reemplazamos v_j por su ecuación

solamente nos va a importar cuando el i que buscamos aparece en la ecuacion y va a quedar por ej $\frac{\partial v_j(n)}{\partial w_{j2}(n)} = \frac{\partial (w_{j2}(n) y_2)}{\partial w_{j2}(n)}$ y derivamos constante por variable y queda solo la constante

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n)$$

Entonces:

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} y_i(n)$$

OJO y_i es la entrada, es decir la salida de la capa anterior y y_j es la salida.

El gradiente de error local instantáneo es:

$$\delta_j(n) = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$$

Local porque es para esa neurona e instantáneo porque es calculado en el instante n.

Para la salida con función de activación sigmoidea

$$y_j(n) = \frac{2}{1+e^{-v_j(n)}} - 1$$

entonces la derivada de la función sigmoide queda:

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \frac{1}{2}(1 - y_j^2(n)) = \frac{1}{2}(1 + y_j(n))(1 - y_j(n))$$

Entonces el gradiente queda:

$$\delta_j(n) = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$$

$$\delta_j(n) = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{1}{2}(1 + y_j(n))(1 - y_j(n))$$

Entonces ahora lo defino de la siguiente manera:

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} y_i(n)$$

$$\frac{\partial \xi(n)}{\partial w_{ij}(n)} = \delta_j(n) y_i(n)$$

$$\Delta w_{ji}(n) = -u \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

$$\Delta w_{ji} = -\, u\, \delta_j(n)\, y_i(n)$$

Retropropagación en capa III (salida)

$$\Delta w_{ji}^{III}(n) = -\, u\, \delta_j^{III}(n)\, y_i^{III}(n)$$

$$\delta_j^{III}(n) = \frac{\partial \xi(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 - y_j(n)^{III})(1 + y_j(n)^{III}) =$$

$$\delta_j^{III}(n) = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 - y_j(n)^{III})(1 + y_j(n)^{III}) =$$

El criterio de error cuadrático medio depende del error cuadrático de la neurona de salida j, a su vez, el error cuadrático de esa neurona j depende de la salida que haya dado esa neurona.

$$\delta_j^{III}(n) = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 - y_j(n)^{III})(1 + y_j(n)^{III}) =$$

como $\xi(n) = \frac{1}{2}\sum_{j=1}^{M} e_j^{\,2}(n)$ $\quad \frac{\partial \xi(n)}{\partial e_j(n)} = e_j(n)$

$$\delta_j^{III}(n) = e_j(n) \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 - y_j(n)^{III})(1 + y_j(n)^{III})$$

$$e_j = d_j^{III}(n) - y_j^{III}(n), \qquad \frac{\partial e_j(n)}{\partial y_j^{III}(n)} = -\,1$$

$$\delta_j^{III}(n) = \frac{1}{2} e_j(n)(1 + y_j(n)^{III})(1 - y_j(n)^{III}) \;\bigstar$$

$$\Delta w_{ji}^{III}(n) = \mu \frac{1}{2} e_j(n)(1 - y_j(n)^{III})(1 + y_j(n)^{III}) y_i^{II}(n)$$

$$\Delta w_{ji}^{III}(n) = \eta e_j(n)(1 - y_j(n)^{III})(1 + y_j(n)^{III}) y_i^{II}(n)$$

Retropropagación en capas ocultas: capa II y capa "p"

$$\Delta w^{II}_{ji} = -\, u\, \delta_j^{II}(n)\, y_i^{I}(n)$$

$$\delta_j^{II}(n) = \frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 - y_j(n)^{II})(1 + y_j(n)^{II})$$

El error ξ es respecto a la capa de salida, pero lo queremos derivar respecto a la capa oculta

$$\frac{\partial \xi(n)}{\partial y_j^{II}(n)} = \sum_k \frac{\partial \xi(n)}{\partial e_k(n)} \frac{\partial e_k(n)}{\partial y_k^{III}(n)} \frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} \frac{\partial v_k^{II}(n)}{\partial y_j^{II}(n)}$$

(hay suma sobre k porque ξ depende de todas las salidas de la capa III)

(el error de la capa de salida respecto a la salida de la capa de salida, la salida respecto a la salida lineal, la salida lineal respecto a la entrada, es decir la salida de la capa oculta). k: neuronas de la capa III (salida). j: neuronas de la capa II (oculta)

como: $v_k^{III}(n) = \sum_j^N w_{kj}^{III} y_j$

$$\frac{\partial v_k^{II}(n)}{\partial y_j^{II}(n)} = w_{kj}^{III}$$

Como ya calculamos antes, para la salida con función de activación sigmoidea $y_k(n) = \frac{2}{1+e^{-v_j(n)}} - 1$

$$\frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)} = \frac{1}{2}(1 - y_k(n)^{III})(1 + y_j(n)^{III})$$

como: $e_k(n) = d_k^{III}(n) - y_k^{III}(n)$

$$\frac{\partial e_k(n)}{\partial y_k^{III}(n)} = -\,1$$

como: $\xi(n) = \frac{1}{2}\sum_k e_k^{\,2}(n)$

$$\frac{\partial \xi(n)}{\partial e_k(n)} = e_k(n)$$

Entonces nos queda

$$\frac{\partial \xi(n)}{\partial y_j^{II}(n)} = \sum_k e_k(n) - 1\frac{1}{2}(1 - y_k(n)^{III})(1 + y_j(n)^{III}) w_{kj}^{III}$$

$$\frac{\partial \xi(n)}{\partial y_j^{II}(n)} = \sum_k -\frac{1}{2} e_k(n)(1 - y_k(n)^{III})(1 + y_j(n)^{III}) w_{kj}^{III}$$

y

$$\delta_j^{II}(n) = \frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 - y_j(n)^{II})(1 + y_j(n)^{II})$$

$$\delta_j^{II}(n) = \sum_k -\frac{1}{2} e_k(n)(1 - y_k(n)^{III})(1 + y_j(n)^{III}) w_{kj}^{III} \frac{1}{2}(1 - y_j(n)^{II})(1 + y_j(n)^{II})$$

De la capa 3 tenemos el gradiente de error local en la capa 3:

$$\delta_k^{III}(n) = -\frac{1}{2} e_k(n)(1 - y_k(n)^{III})(1 + y_k(n)^{III})$$

Entonces la fórmula del gradiente del error local instantáneo de la capa 2 nos queda:

$$\delta_j^{II}(n) = \delta_k^{III}(n) w_{kj}^{III} \frac{1}{2}(1 - y_j(n)^{II})(1 + y_j(n)^{II})$$

Estamos haciendo pasar el gradiente de error local de la capa 3 de atrás para adelante, de la salida hacia la capa oculta, aquí se ve el concepto de retropropagación. Se está pasando el gradiente del error de la capa 3 por los pesos de la capa 3 para obtener el gradiente de la capa 2, a diferencia de pasar las entradas por los pesos cuando hacíamos propagación hacia adelante.

Esto se puede generalizar para la capa "p":

$$\Delta w_{ji}^{(p)}(n) = -\mu \delta_j^{(p)}(n) y_i^{(p-1)}(n)$$

Con

$$\delta_j^{(p)}(n) = (\delta^{(p+1)} \star w_j^{(p+1)}) \frac{1}{2}(1 - y_j^{(p)}(n))(1 + y_j^{(p)}(n))$$

p es la capa en la cual estamos parados actualmente, la capa para la cual queremos calcular el ajuste de pesos.
p-1 será la entrada a esa capa
μ velocidad de aprendizaje
$(\delta^{(p+1)} \star w_j^{(p+1)})$ es el producto interno que es la retropropagacion del gradiente de error local de la capa siguiente p+1
$(1 - y_j^{(p)}(n))(1 + y_j^{(p)}(n))$ es la derivada de la función de activación de la capa p

$y_i^{(p-1)}(n)$ es la entrada a la capa p, la salida de la capa p-1

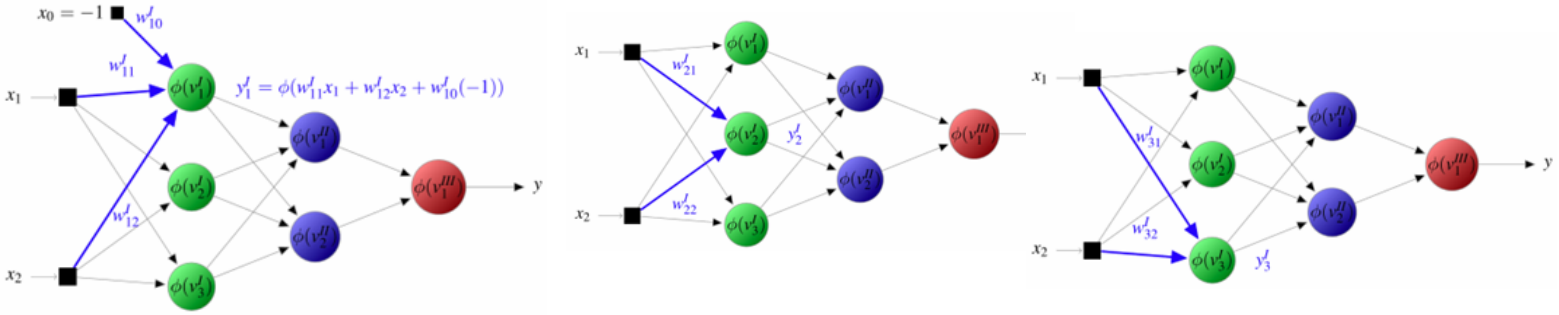
Resumen del algoritmo de retropropagación

1. Inicialización aleatoria
2. Propagación hacia adelante
3. Propagación hacia atras
4. Adaptación de los pesos
5. Iteración: vuelve a 2 hasta convergencia o finalización

Todos estos pasos se van a repetir para cada patrón.

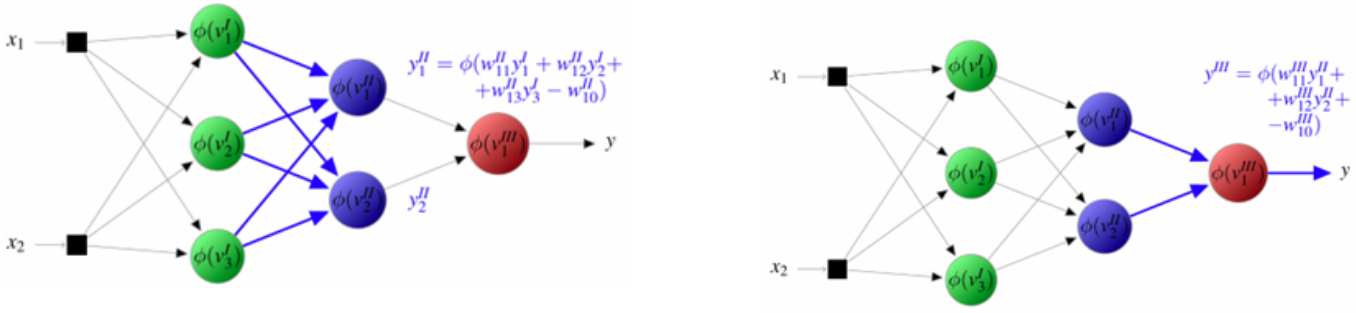
Propagación hacia adelante

Capa 1:



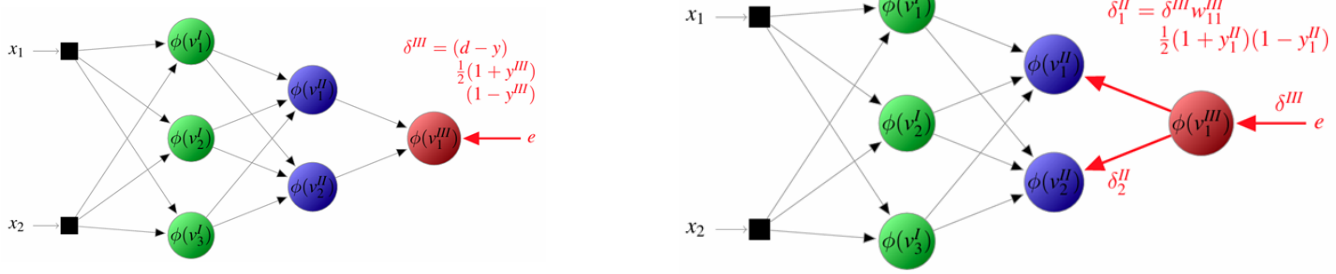
Capa 2 y capa 3:

Recordar que también se suma el sesgo que no está graficado

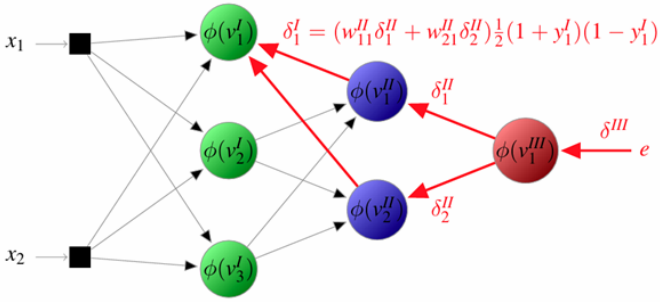


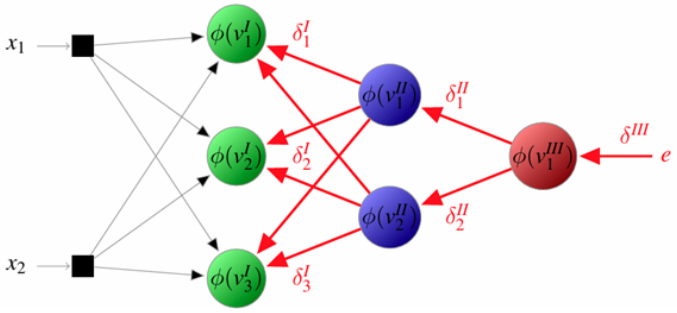
Propagación hacia atrás

Capa 3 y capa 2:



Capa 1:

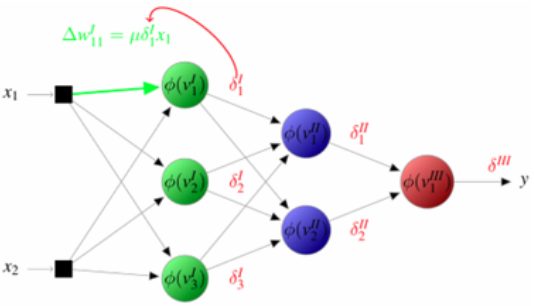


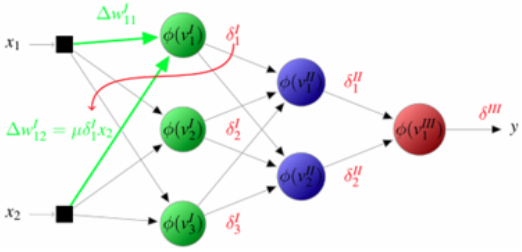


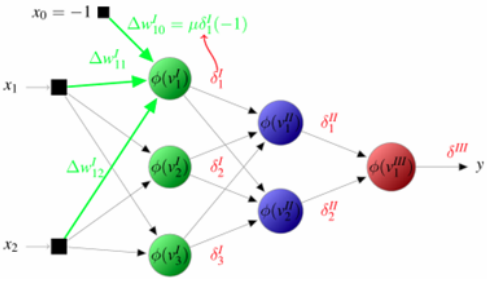
Ajuste de pesos

El delta que calculo, luego se debe sumar al peso actual

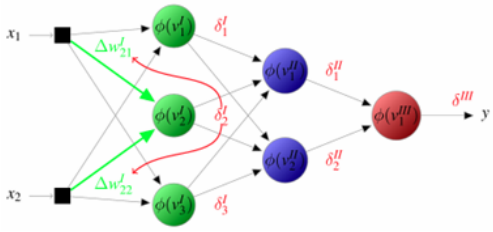
Capa 1: Neurona 1:

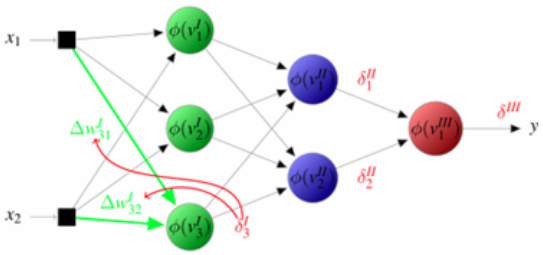




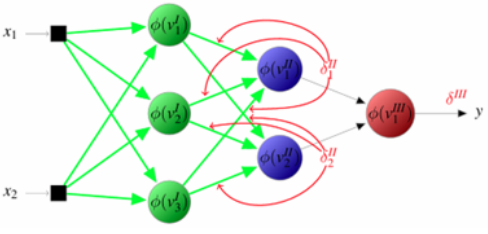


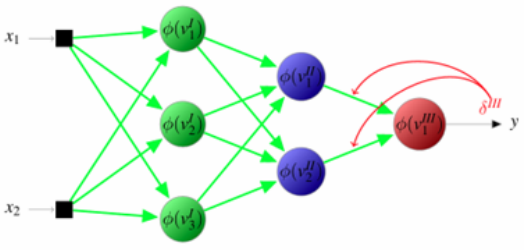
Capa 1: Neurona 2 y 3





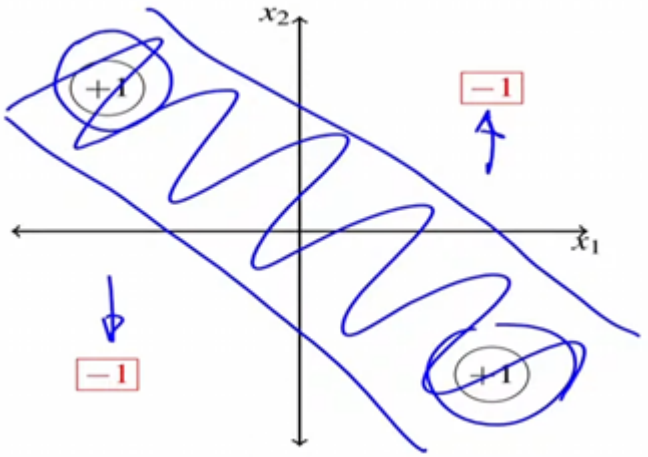
Capas 2 y 3:





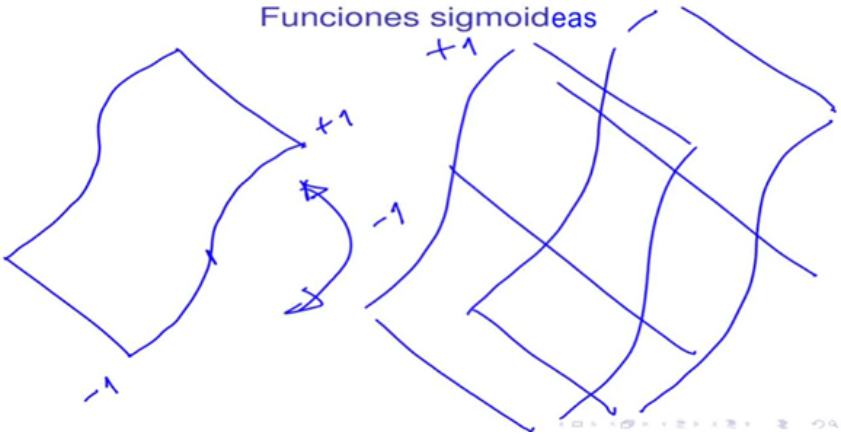
Redes con Funciones de Base Radial (RBF-NN)

Con la función signo o sigmoidea, se logra la siguiente separación mediante fronteras de decisión:



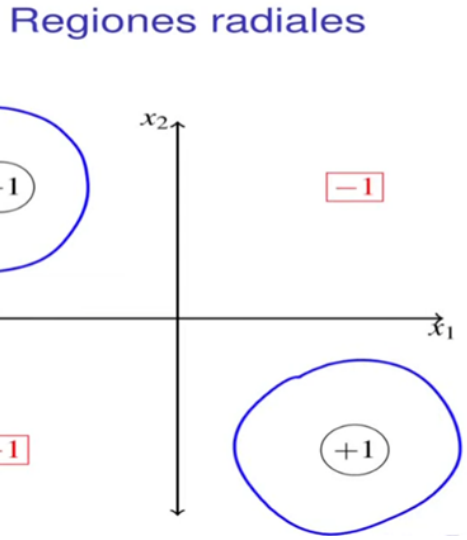
Estas fronteras de decisión son donde la función de activación pasa de -1 a +1.

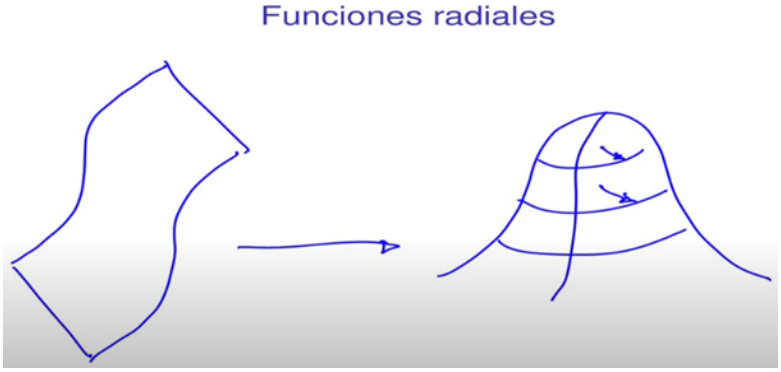
En 3d las sigmoideas se ven asi:



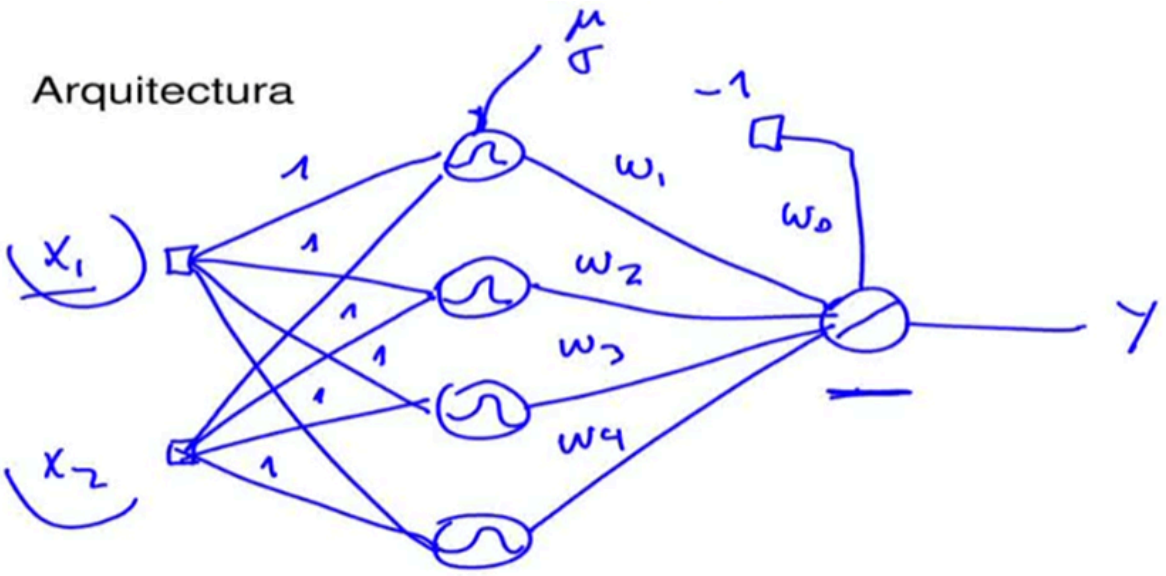
En cada recta de la región separada con dos perceptrones habría una función sigmoidea (imagen de la derecha).

Si en vez de usar las funciones sigmoidea o signo usamos una radial que encierre la zona que nos interesa con un círculo podríamos clasificar todo lo que esté dentro del círculo como una clase y todo lo que esté afuera como otra.





RBF - NN Arquitectura



En este ejemplo, las gausseanas están en R^2 porque reciben x1 y x2.

Las neuronas intermedias son gausseanas y la de salida es lineal, esto porque simplemente es para aproximación de funciones, los pesos para las capa de entrada son 1. Las neuronas de base radial no requieren bias.

La relación entre las entradas y las gausseanas se ajustan con la media y la desviación estándar. Cada valor es un vector de 2 valores porque hay dos entradas.

RBF-NN Modelo Matemático

Modelo matemático

$$y_k(\mathbf{x}_\ell) = \sum_{j=1}^{M} w_{kj}\phi_j(\mathbf{x}_\ell)$$

El subíndice k indica la neurona que estamos, k porque pueden ser varias salidas

para cada salida tenemos su patron deentrada l

cada neurona va a tener un peso asociado j y con la sumatoria recorremos los pesos

donde:

$$\phi_j(\mathbf{x}_\ell) = e^{-\frac{\|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2}{2\sigma_j^2}}$$

xl - uj -> distancia entre la entrada y el vector de medias. estamos viendo la entrada que tan lejos de la media de cada una de las gaussianas

j indice de cada una de las neuronas de la capa radial

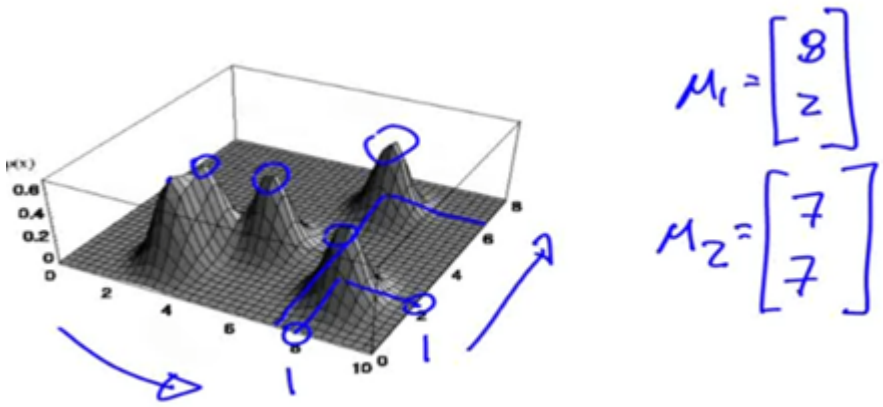
μ_j es el vector de medias de la neurona j, Equivale al centroide.

σ_j es la desviación estándar de la neurona j. Equivale al "ancho" de la gaussiana. Aca está simplificado porque es un solo valor, con esto es suficiente porque pesos de la capa de salida hacen que le dé más o menos importancia a esa gaussiana, por lo que me evita calcular las desviaciones para que sea más o menos ancha. Si quiero cambiar la forma también, debo usar gaussianas N-dimensionales.(no lo veo)

La salida y_k va a ser de un perceptrón simple con salida lineal.

La función depende de la distancia entre la entrada y el centro.

Cada neurona j de base radial va a tener su propio vector de medias.



RBF - NN Entrenamiento

Se adaptan las redes con funciones de base radial y los pesos w_{kj} por separado

- **Método 1:**
  - **Paso 1:** Adaptación no supervisada de las RBF
    Primero se entrena sin decirle cuales son las salidas correctas
    - Utilizando k-medias
    - Utilizando mapas auto organizativos
    - Otros
  - Paso 2: Adaptación supervisada de los w_{kj} (LMS)
    Comparando con las salidas correctas se adapta SÓLO los pesos

- **Método 2:**

En este método además se hace una adaptación supervisada tanto de los pesos sinápticos como de las medias y desvíos

- **Paso 1:** Inicialización por el método 1
- **Paso 2:** Adaptación supervisada de las RBF $(\frac{\partial \xi}{\partial \mu_{ji}}, \frac{\partial \xi}{\partial \sigma_j})$

NO SUPERVISADO: quiere decir que no se utiliza la salida deseada para el entrenamiento.

Vamos a dar mas detalle del método 1

1 Adaptación de las RBF

Utilizando k-medias

Método NO-supervisado! (uno de los más simples)

Objetivos:

- Encontrar k conjuntos C_j de forma que:
  - Cada conjunto C_j sea lo más diferente posible de los demás
  - Los patrones $\mathbf{x}_\ell$ dentro de cada C_j sean lo más parecidos posible entre ellos
- Encontrar el centroide $\boldsymbol{\mu}_j$ de cada conjunto C_j

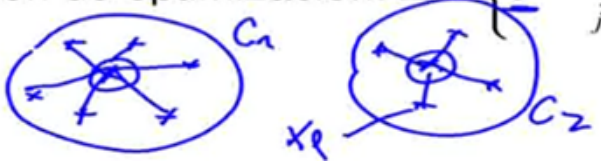
Ecuación de optimización: $\min \left\{ J = \sum_{j=1}^{k} \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 \right\}$

k y Cj están RELACIONADOS (j neuronas de la capa oculta/radial). Si tengo 10 neuronas en la capa radial voy a tener que buscar 10 conjuntos 1 para cada uno de las neuronas.

Los centroides van a ser luego, las medias de esas gaussianas. Son a su vez el promedio de todos los patrones de cada conjunto.

Se minimiza las distancias entre los centroides y los patrones:

Ecuación de optimización: $\min \left\{ J = \sum_{j=1}^{k} \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 \right\}$



Acumulación de las distancias entre todos los centroides y sus patrones. Minimizamos esto para ubicar el centroide bien en el medio.

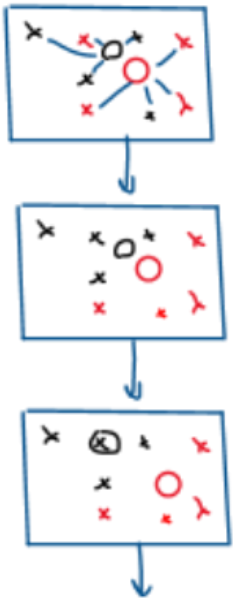
1. Inicialización: se forman los k conjuntos $C_j(0)$ con patrones $\mathbf{x}_\ell$ elegidos al aleatoriamente.
2. Se calculan los centroides:

$$\boldsymbol{\mu}_j(n) = \frac{1}{|C_j(n)|} \sum_{\ell \in C_j(n)} \mathbf{x}_\ell$$

3. Se reasignan los $\mathbf{x}_\ell$ al C_j más cercano:

$$\ell \in C_j(n) \Leftrightarrow \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 < \|\mathbf{x}_\ell - \boldsymbol{\mu}_i\|^2 \quad \forall i \neq j$$

4. Volver a 2 hasta que no se realicen reasignaciones.



Procedimiento opción 1: k-medias por lotes

x_l y u_j *vectores porque están en negrita*

$$\ell \in C_j(n) \Leftrightarrow ||x_l - \mu_j||^2 < ||x_l - \mu_i||^2 \; \forall i \neq j$$

el l-ésimo patrón pertenece al conjunto j Cj si y sólo si la distancia entre el l-ésimo patrón y el centroide del conjunto j es menor que la distancia entre el patrón y los centroides de todos los demás conjuntos posibles.

LA DISTANCIA QUE YO ELIJO TIENE QUE SER MENOR A LA DISTANCIA QUE TIENE A TODOS LOS DEMÁS

k es la cantidad de neuronas que tenemos en la capa de base radial.

El paso 2 es un promedio simplemente, toma los patrones del conjunto j, los suma y los divide por la cantidad de elementos que tiene el conjunto j. X es un vector.

Este método hace el proceso por lotes, es decir, hacer con todos los patrones, ajustar los centroides, hacer con todos los patrones, ajustar los centroides.

Procedimiento opción 2: k-medias online

Se realiza patrón a patrón:

Se calcula el valor del centroide en la iteración siguiente en base a el valor actual, el patrón que acaba de entrar y una velocidad de aprendizaje.

1. Inicialización: se eligen k patrones aleatoriamente y se usan como centroides iniciales $\boldsymbol{\mu}_j(0) = \mathbf{x}'_\ell$.
2. Selección:

$$j* = \arg\min_j \left\{ \|\mathbf{x}_\ell - \boldsymbol{\mu}_j(n)\| \right\}$$

3. Adaptación:

$$\boldsymbol{\mu}_{j*}(n+1) = \boldsymbol{\mu}_{j*}(n) + \eta(\mathbf{x}_\ell - \boldsymbol{\mu}_{j*}(n))$$

4. Volver a 2 hasta no encontrar mejoras significativas en J.

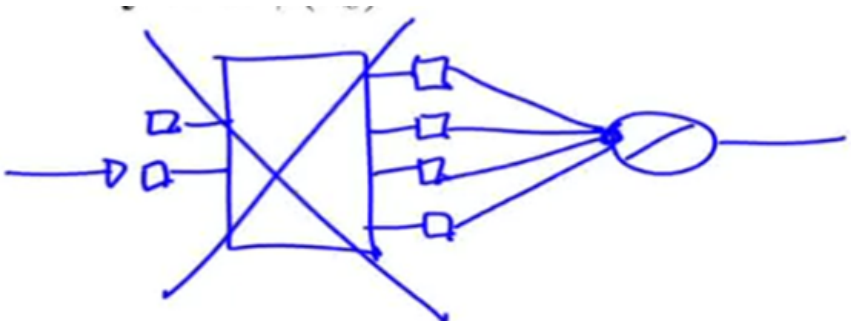
2: entra un patrón y se elige el centroide ganador con la menor distancia al patrón que acaba de entrar.

3: se adapta el peso de ese centroide ganador hacia el patrón. es un vector que apunta desde el centroide al patrón.

2 Adaptación de los pesos

- Al entrenar los pesos, las RBF quedan fijas
- Al estar las RBF fijas se pueden obtener las salidas intermedias para cada patrón de entrada: $\phi(\mathbf{x}_\ell)$
- Con esas salidas intermedias se puede entrenar cada perceptrón simple:

$$\mathbf{y} = \mathbf{W}\phi(\mathbf{x}_\ell)$$



esta simplificación nos permite olvidarnos de las primeras entradas y tomar estas segundas como las entradas.

Método de entrenamiento LMS: Gradiente descendiente sobre el error cuadrático instantáneo

$$w_{kj}(n+1) = w_{kj}(n) - \eta \frac{\partial \xi(n)}{\partial w_{kj}(n)}$$

$$\frac{\partial \xi(n)}{\partial w_{kj}(n)} = \frac{\partial \xi(n)}{\partial e_k(n)} \frac{\partial e_k(n)}{\partial y_k(n)} \frac{\partial y_k(n)}{\partial w_{kj}(n)}$$

$y_k(n) = \sum_j w_{kj}(n)\phi_j(n)$ *entonces*

$$\frac{\partial y_k(n)}{\partial w_{kj}(n)} = \phi_j(n)$$

$e_k(n) = d_k(n) - y_k(n)$ *entonces*

$$\frac{\partial e_k(n)}{\partial y_k(n)} = -1$$

$\xi(n) = \frac{1}{2}\sum_k e_k^2(n)$ *entonces*

$$\frac{\partial \xi(n)}{\partial e_k(n)} = e_k(n)$$

$$w_{kj}(n+1) = w_{kj}(n) - \eta e_k(n)\phi_j(n)$$

Comparación RBF-NN vs. MLP

RBF-NN	MLP
1 capa oculta	p capas ocultas
distancia a prototipos gaussianos	hiperplanos sigmoideos
representaciones locales sumadas	representaciones distribuidas combinadas
convergencia más simple (linealidad)	
entrenamiento más rápido	
arquitectura más simple	
combinación de diferentes paradigmas de aprendizaje	

1 capa oculta porque si vamos agregando gaussianas podemos resolver cualquier tipo de problemas, puede partir el espacio con las gaussianas ubicandolas donde quiera formando regiones muy complejas

Redes Neuronales dinámicas

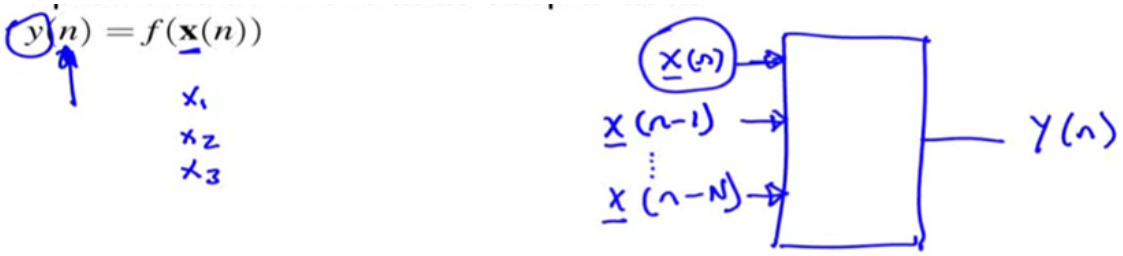
Son redes que tienen un comportamiento que evoluciona con el tiempo.

Caso general:

$$y(n) = f(\mathbf{x}(n), \mathbf{z}_1(n), \mathbf{y}_1(n))$$

Tiene realimentación de las entradas en instantes anteriores, de los estados internos en instantes anteriores y de las salidas en instantes anteriores:

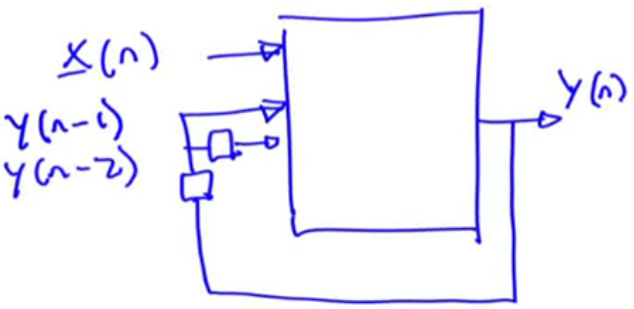
- **Aproximación 1: entradas desplazadas**



n es el tiempo discreto (iteración)
Con esta aproximación suponemos que en una red neuronal hay entradas que no solo contemplan el momento actual x(n) sino lo que valieron esas mismas entradas en instantes anteriores x(n-N).
Estas entradas anteriores también influyen en la salida. las x son vectores
La red sigue siendo estática, ya que no hay una dinámica interna, solo le pasó entradas en instantes anteriores.

- **Aproximación 2: realimentación de las entradas**

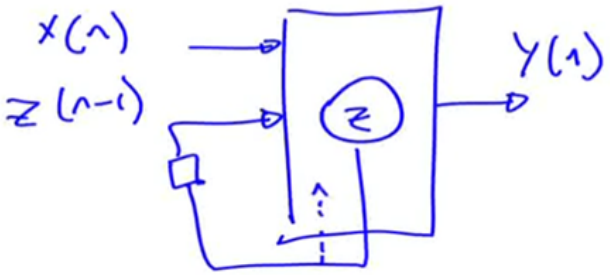
$$y(n) = f(\mathbf{x}(n), \mathbf{y}_1(n))$$



  Además de tener las entradas, tengo las salidas en instantes anteriores como entradas.

- **Aproximación 3: realimentación de estados internos**

$$y(n) = f(\mathbf{x}(n), \mathbf{z}_1(n))$$



Se realimenta un estado interno de un instante interior. Hasta acá sigue siendo TDNN, estática.
Si se realimenta a alguna capa interna de la red neuronal (con retardo), es decir sin pasar como entrada al principio (flecha discontinua), En este caso la red neuronal en sí misma si se comporta de forma dinámica:
La salida actual va a depender no solo de la entrada actual sino de los estados anteriores de la red es decir de las salidas internas anteriores, que indirectamente dependen de las entradas anteriores de la red. La red empezó a considerar cierta memoria, responde a la historia de las entradas.

Clasificación

Redes neuronales dinámicas (DNN)

- Redes con retardos en el tiempo (TDNN).
  Generalización de las redes que planteamos anteriormente (los

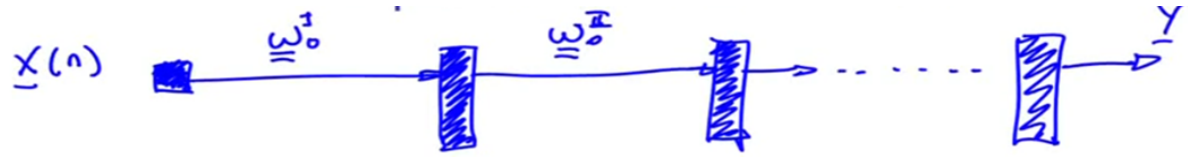
bloques). Tienen un comportamiento dinámico, pero se pueden seguir entrenando con los métodos que ya conocemos. Siguen siendo feed forward.

- **Redes recurrentes (RNN)**
  Tienen realimentación hacia atrás. La sinapsis no va siempre hacia adelante, sino que hay realimentación de una neurona a sí misma, a otra de la misma capa o a una neurona de la capa anterior.
  - Redes totalmente recurrentes
    - Redes de Hopfield (memorias asociativas) ß solo vemos esta
    - Redes de Boltzman (supervisadas)
    - Teoría de la resonancia adaptativa (ART)
  - Redes parcialmente recurrentes (PRNN)
    - Método de retropropagacion a través del tiempo (BPTT) ← solo vemos esto
    - Redes de Elman ← solo arquitectura
    - Redes de Jordan ← solo arquitectura

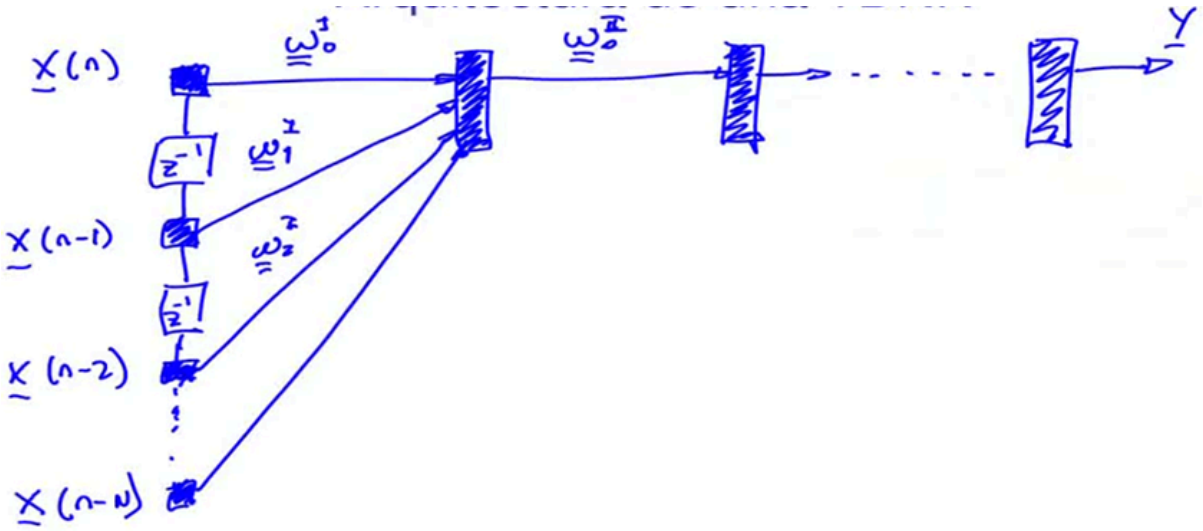
Redes neuronales con retardos en el tiempo TDNN

Arquitectura de una TDNN

Arranca como un perceptrón multicapa, el _0 de los pesos quiere decir con retardo 0.



Se agrega un retardo temporal y las mismas entradas, pero en un instante anterior (n-1). Esto se agrega como entrada en la capa 1 con una matriz de pesos, pero con retardo 1. Esto se puede expandir hacia la cantidad de retardos N que queramos



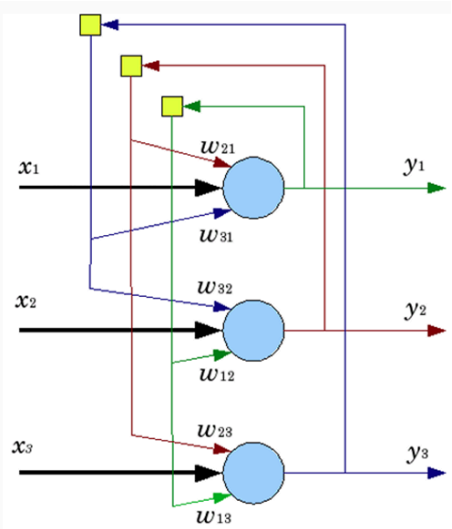
Puede ser entrenada con retropropagación como el perceptrón multicapa, solo que cambiando el formato de los pesos para tener en cuenta que paso en el tiempo.

Redes recurrentes RNN

Totalmente recurrentes

Redes de hopfield

Arquitectura



En estas redes siempre hay la misma cantidad de entradas, neuronas y salidas y hay una sola capa.

Se realimenta la salida de una neurona con un retardo temporal (el cuadradito) al resto de las neuronas. No hay realimentaciones a la misma neurona.

Los pesos sinápticos están vinculados a las salidas de las neuronas y las realimentan como entradas.

Modelo matemático

$$y_j(n) = sgn(x) = sgn\left(\sum_{i=1}^{N} w_{ji} y_i(n-1) - \theta_j\right) \cdots \begin{cases} x > 0 & +1 \\ x = 0 & y_j(n-1) \\ x < 0 & -1 \end{cases}$$

$$w_{ji} = w_{ij} \quad \forall\, i \neq j$$
$$w_{ii} = 0 \quad \forall i$$

$w_{ji} y_i(n-1) \rightarrow$ *Sumatoria de los pesos por la salida del instante anterior*

$\theta_j \rightarrow$ *Sesgo*

Estas redes son binarias, es decir que dan +1 o -1 en la salida y solo pueden recibir +1 y -1 en las entradas

x llamamos la sumatoria en este caso. Si la sumatoria es 0, la salida permanece igual que en el instante anterior

Los pesos sinápticos son simétricos: el peso de la salida de la neurona 1 que vincula con la neurona 2 es lo mismo que el peso que los vincula al revés. Y no hay pesos que vinculen a una neurona consigo misma (no hay realimentación en una neurona).

Generalidades

- **Cada neurona tiene un disparo probabilístico**
  Es decir que hay una variable aleatoria que se elige al azar a cuál le toca disparar en cada momento.
- **Conexiones simétricas**
- **El entrenamiento es no-supervisado**
- **Puede utilizarse como memoria asociativa**
  Es decir que se puede utilizar no como memoria por dirección, donde se le pasa la dirección y te devuelve el contenido, sino que se utiliza de manera que le doy algo que quiero encontrar y me devuelve lo más parecido que tenga en memoria. Es decir, la puedo acceder por el contenido y no por una dirección. Por ejemplo le doy la foto de una persona y me da la cara en limpio.

Almacenamiento y recuperación (Entrenamiento y prueba)

Entrenamiento (almacenamiento)

Dado un conjunto de patrones (memorias fundamentales o datos limpios): Por ej datos de un rostro

$X^* = \{x_k^* \epsilon \mathbb{R}^N\}$

Tengo P patrones que quiero guardar en la red, cada patrón x_k^* es un es un vector de N componentes (una memoria de N neuronas). Estos patrones son las memorias fundamentales que la red podrá recordar.

Para calcular los pesos sinápticos de la red se utiliza Aprendizaje Hebbiano:

$$w_{ji} = \frac{1}{N} \sum_{k=1}^{P} x_{kj}^* x_{ki}^*$$

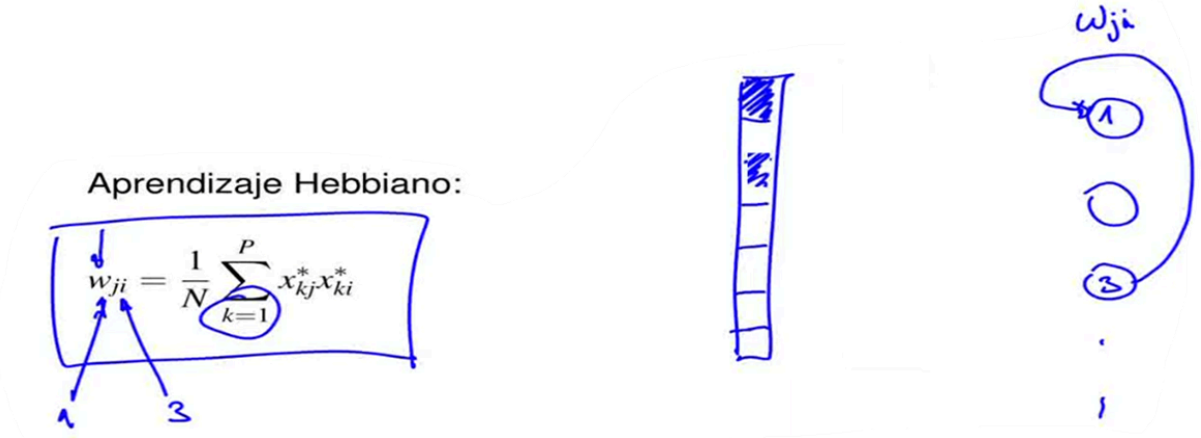
Hace una especie de promedio de los patrones. El peso que vincula el la entrada con la . Promedia la i y la j para todos los patrones.

Calcula la relación o sincronización entre dos neuronas, si no tienen sincronización, el peso será 0. Cuando las neuronas están sincronizadas el peso sináptico aumenta.

¿Cómo se interpreta esto?:

- Estamos tratando de averiguar qué pasa con la conexión que hay la neurona i y la j
  - Si ambas siempre son -1 o ambas 1 → el peso se hace positivo y grande
  - Si uno siempre es -1 y otro +1 → el peso se hace negativo y grande
  - Si algunas veces son +1 y -1 o -1 y +1 → el peso va a ser cercano a cero.

Por ejemplo, dada la entrada 1 y la entrada 3 de muchos patrones:



En la imagen se ve el patron k.

Si en la mayoría de patrones, ambos valen +1, el peso es muy grande y positivo, si ambos valen -1 también el peso será grande. Si uno vale +1 y el otro -1 en la mayoría de los casos, va a dar

-1. Esto quiere decir que esas dos neuronas están sincronizadas a la inversa, por lo que el peso va a ser negativo y grande.

Entonces se puede observar que si el peso es grande y positivo hay sincronización, si es negativo y grande hay sincronización inversa y si no hay sincronización el peso será 0. A veces las dos están en +1, a veces en -1, a veces cruzadas, etc.

Observaciones

- **El proceso de entrenamiento NO es interactivo.** Le mostramos una vez todos los patrones, se realiza el cálculo y termino.
- w_{ij} es mayor cuando las neuronas i, j se tienen que activar juntas (regla de Hebb).
- La capacidad de almacenamiento está limitada a: $P = \frac{N}{2ln(N)}$ con un 1% de error. Esta es la cantidad máxima de memorias fundamentales que podemos almacenar. P es la cantidad máxima de memorias fundamentales, por ej queremos almacenar 200 rostros serian 200 memorias fundamentales, esta limitada por la cantidad de neuronas

Prueba (recuperación)

Dado un patrón x(incompleto, ruidoso…) se fuerza:

$$y(0) = x$$

Esto es inicializar la salida en el tiempo 0 con la entrada. La primera salida es el patrón que queremos buscar.

Iteración

1. Se elige al azar una neurona
   $j \mathrel{*}= rnd(N)$

2. Para esa neurona calculamos esa salida
   $$y_j \mathrel{*}= sgn(\sum_{i=1}^{N} w_{ji} y_i(n-1)$$

3. Volver hasta 1 hasta no observar cambios en las y_j

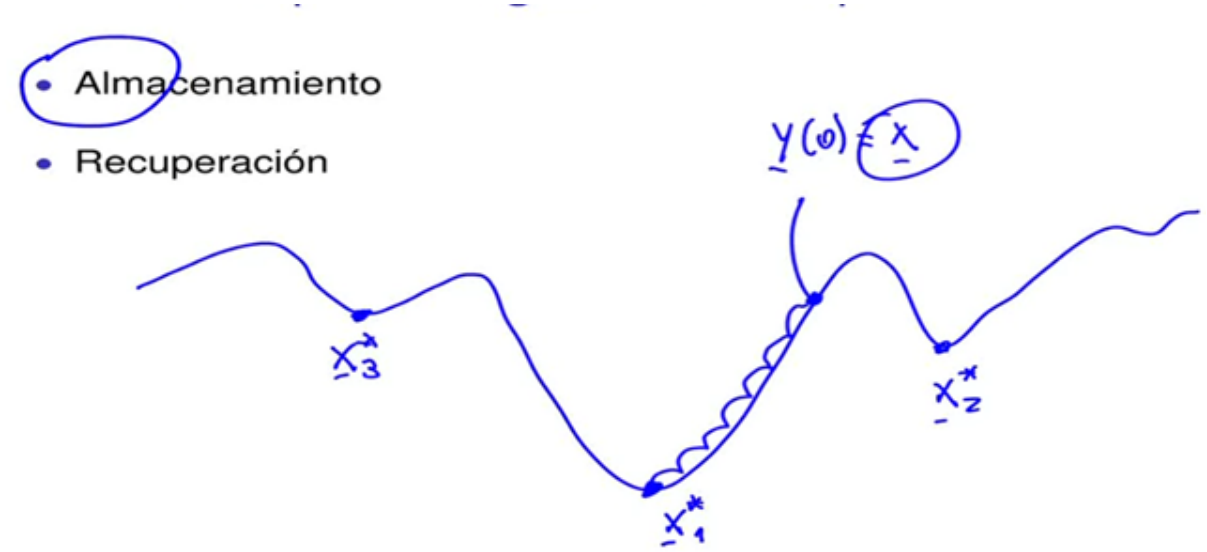
Cuando se cumple que no hay cambios, se convergió a una memoria o patrón fundamental. Es decir, el valor de las salidas es una de las memorias fundamentales almacenadas.

Observaciones

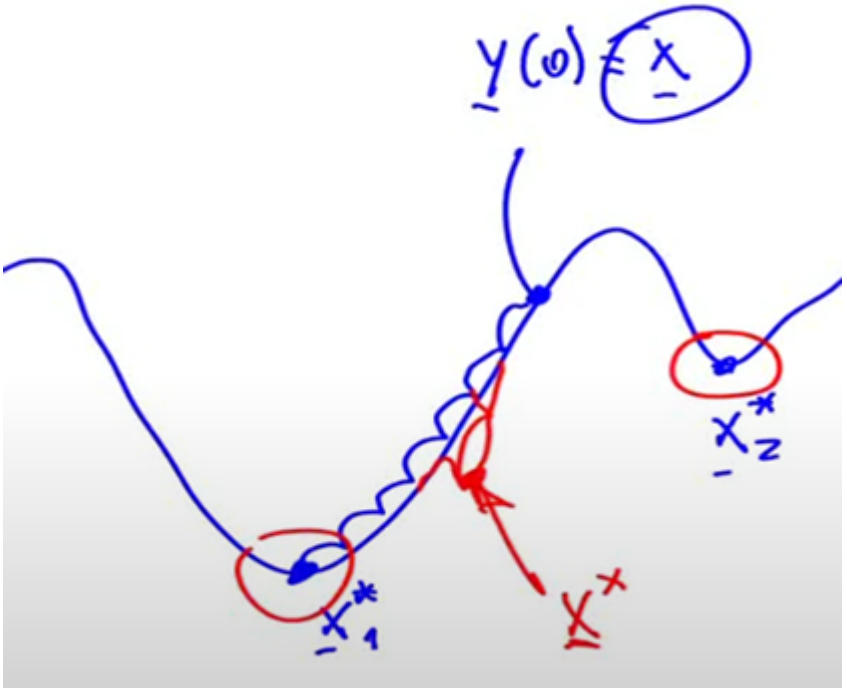
- **El proceso de recuperación ES iterativo (dinámico)**
  La dinámica es que se calcula y se itera hasta estabilizarse y recién ahí dan una salida. No es que entra algo y sale algo.
- En general no se utilizan los θ_j
- La salida final es cuando no hay cambios al "recorrer" todas las salidas

- Se pueden obtener estados espurios y oscilaciones
  Esto quiere decir que no converge. Esto puede suceder cuando queremos almacenar más memorias fundamentales que las posibles o cuando hay dos memorias fundamentales parecidas.

El almacenamiento y recuperación se puede graficar mediante los campos energéticos de Hopfield:



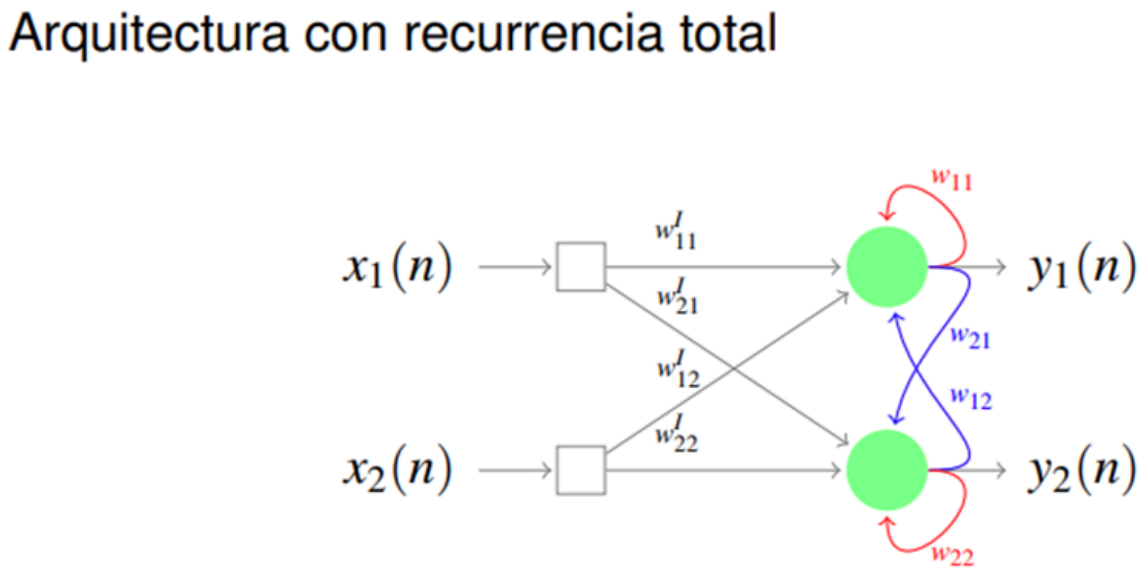
En almacenamiento se generan estos "valles". En recuperación se itera hasta encontrar la memoria fundamental.
Si no estuvo bien entrenado o superamos la cantidad de memorias fundamentales máximas se genera un valle espureo, en la recuperación se cae en un valle de una memoria que no es fundamental:



Parcialmente recurrentes (PRNN)

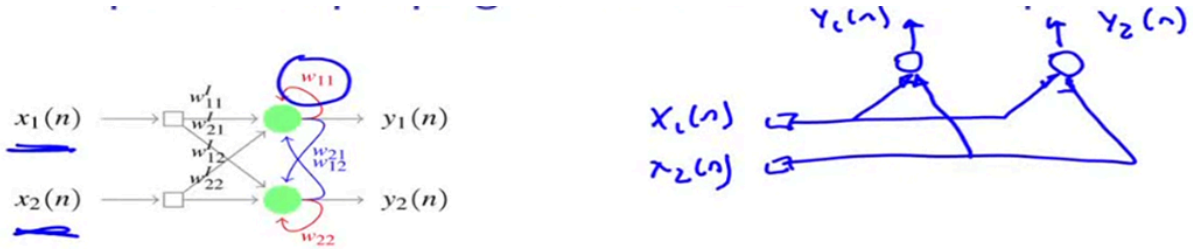
Retropropagación a través del tiempo (BPTT)

Es la forma de convertir redes recurrentes en parcialmente recurrentes y de esa manera expandirla solo para que tenga conexiones hacia adelante.

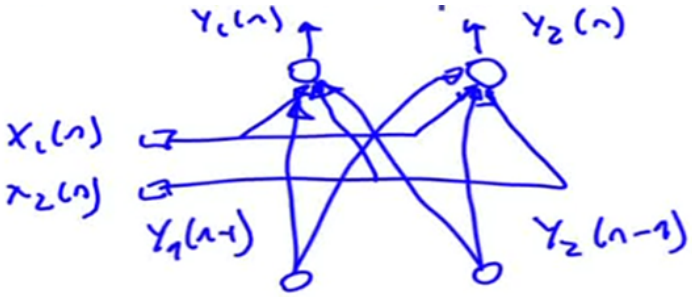


Hay pesos entre las neuronas todas con todas y con sí misma. Hay recurrencias totales.

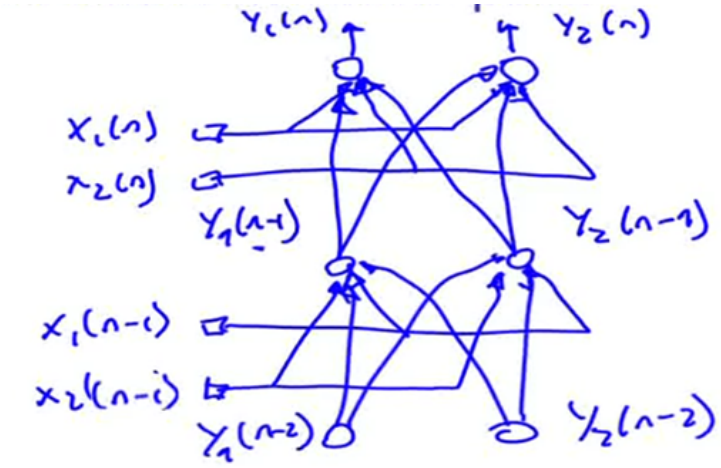
Para entrenar estas redes se expande la red neuronal a lo largo del tiempo:



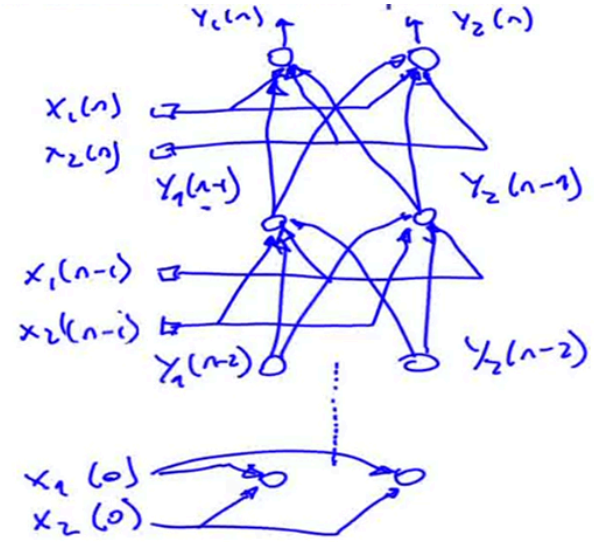
Las neuronas tienen las entradas conectadas a cada una, pero también tenemos como entrada las salidas de ellas mismas en un instante anterior, por lo para calcular las salidas en el instante n se expande:

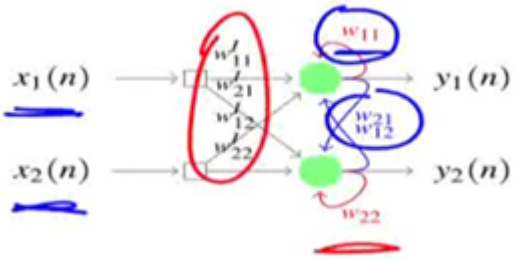


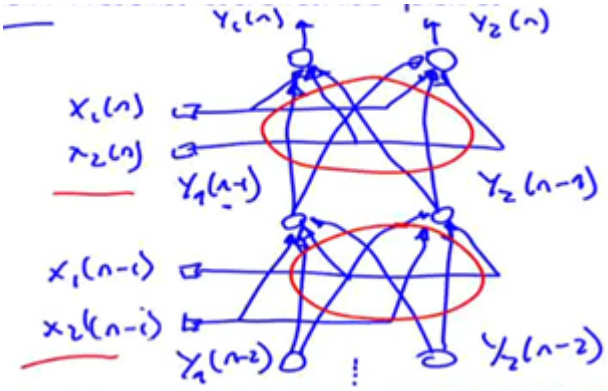
Ahora para calcular la salida en el instante n-1 agrego las entradas, pero para calcular esas salidas también necesito las salidas de las neuronas en dos instantes anteriores:



Esto se hace hasta el instante inicial donde se inicializan las salidas con los valores de x:



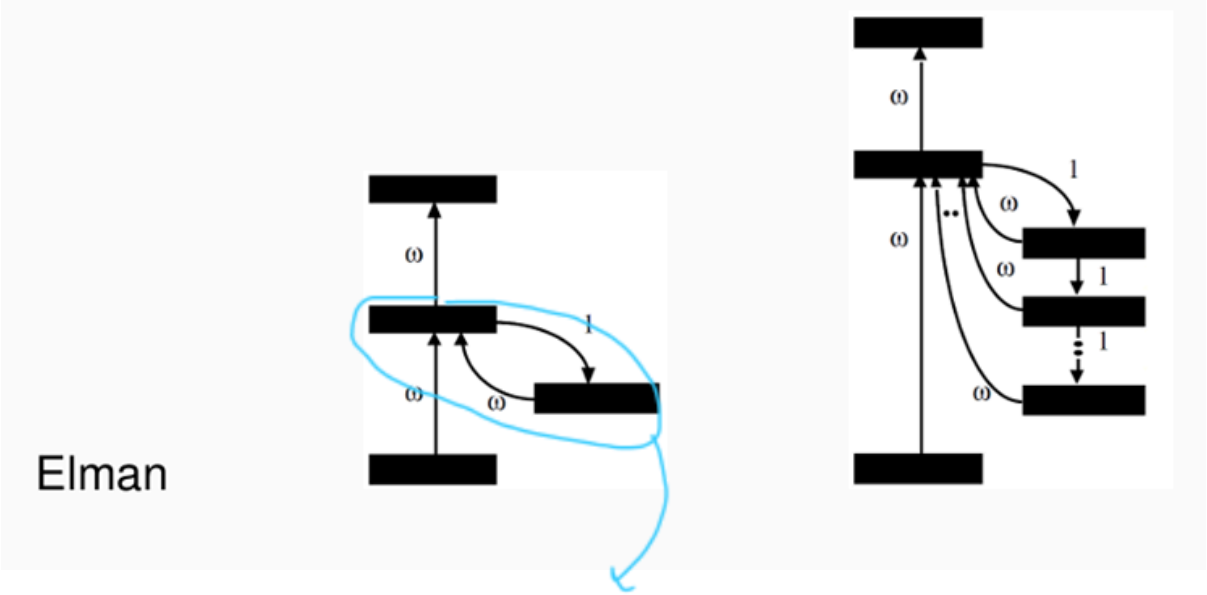




Esta expansión hizo que no existan retropropagaciones y que la propagación sea puramente hacia adelante, por lo que se puede entrenar como entrenamos cualquier perceptrón multicapa. OJO hay que tener en cuenta que los pesos de la primer capa serían los mismos que de la segunda (círculo rojo en gráfico derecha), por lo que se tendría que hacer algún promedio luego de la propagación hacia atrás del entrenamiento. Estos pesos son los 8 del grafiquito, los w' y los w.

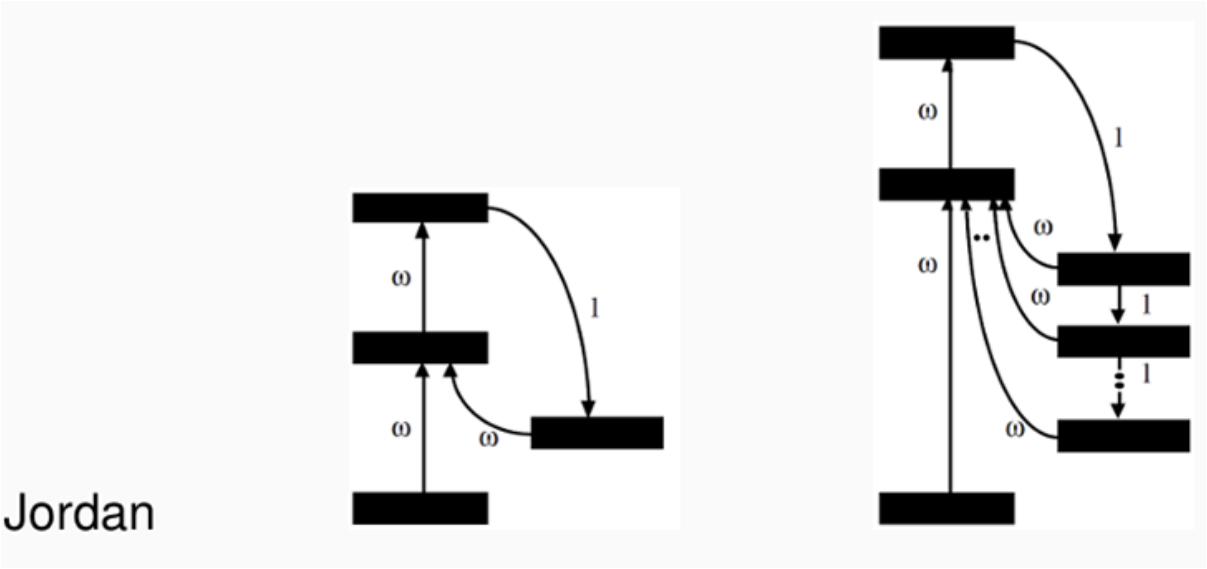
Se puede hacer retropropagación truncada diciendo solo quiero ir hasta dos instantes anteriores.

Elman y Jordan



Hay un retardo de la capa intermedia hacia si misma. La salida de la capa oculta se guarda y en el instante de tiempo siguiente se vuelve a ingresar en esa capa.

A diferencia de las TDNN, estas no son feed forward, sino que hay una realimentación.



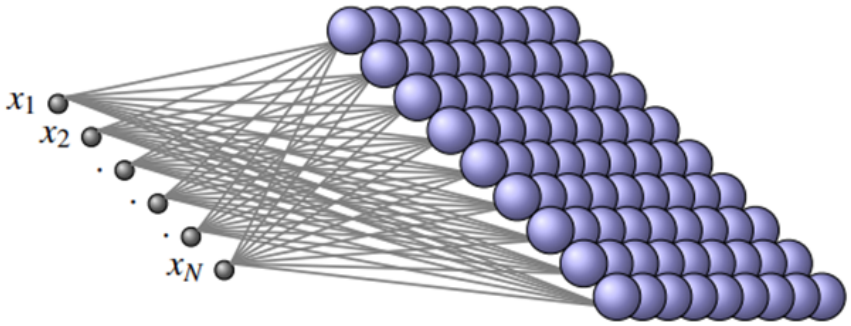
A diferencia de Elman, la realimentación se toma de la salida de la red.

Mapas Autoorganizativos (SOM)

Es el proceso en el cual mediante de interacciones locales se obtiene un ordenamiento global

- Ejemplo 1: evolución de la ubicación de los alumnos asisten a un curso...
- Ejemplo 2: las células del cerebro se auto-organizan en grupos de acuerdo a la información que reciben...

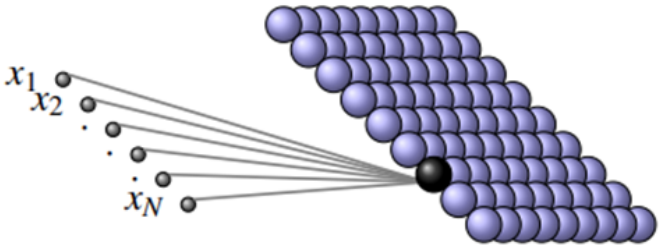
SOM: Arquitectura



Las neuronas están organizadas en un plano, una entrada se conecta con todas las neuronas mediante un peso. Tenemos vector de entradas, matriz de pesos y capa de neuronas.

- Arquitectura SOM 2D

$$\mathbf{x} \rightarrow \mathbf{w}_j \rightarrow s_j$$



Cada entrada tiene un peso asociado con la neurona .

Es una arquitectura competitiva, es decir solo se activa una neurona.

w y x vectores

- Salidas: se activa sólo la ganadora j^*

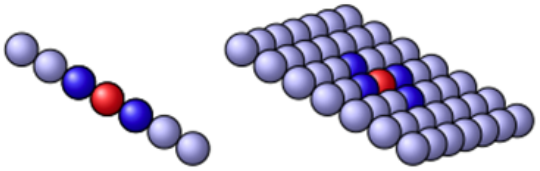
$$j^*(n) = G(\mathbf{x}(n)) = \arg\min_j \{\|\mathbf{x}(n) - \mathbf{w}_j(n)\|\}$$

Se compara la entrada x con todo el vector de pesos y aquel peso que se parece más a la entrada es el de la neurona ganadora.

POR ESO EL ÍNDICE QUE BARRE ES EL j BUSCAMOS EL MÍNIMO RESTANDO LA ENTRADA CON TODOS LOS PESOS

Entornos de influencia o vecindades

- Formas básicas:





Una neurona activada puede estimular a neuronas cercanas. Las neuronas se pueden organizar en una cuadricula, una línea, o en una arquitectura hexagonal.
Estos entornos pueden ser (alcance Λ_G):

- De tamaño fijo
- De tamaño variable

La excitación de las neuronas del entorno con respecto a la ganadora puede ser:

- Excitatorias puras
- Inhibitorias puras

Funciones generales de excitación/inhibición

$h_{G,i}$ es la función de influencia en el entorno. G es la neurona ganadora, i es cualquier otra neurona en la que preguntamos: ¿esta neurona se ve afectada por la ganadora? y puede ser:

- Uniforme (entorno simple)

por ej para entorno $\Lambda_G = 2$

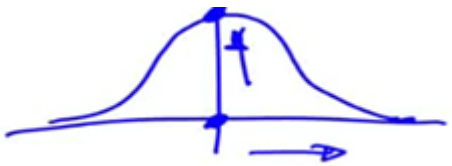
$$\Lambda_G = 2 \Rightarrow \begin{cases} h_{G,i} = \beta(n) & \text{si } |G-i| \leq 2 \\ h_{G,i} = 0 & \text{si } |G-i| > 2 \end{cases}$$

|G-i| Esto quiere decir que si la distancia entre la neurona ganadora y cualquier neurona es menor o igual a 2, se excita con la constante beta, caso contrario la función de exitacion/inhibición da 0, por lo que no está afectada.

$\beta(n)$ *es una funcion que devuelve un valor distinto segun la iteracion que estemos*
por ej:
$n = 2 \rightarrow \beta(2) = 3$
$n = 10 \rightarrow \beta(10) = 1$

$n = 100 \rightarrow \beta(100) = 0.1$

Gaussiana: Es decir, cuanto más lejos menos se excita.



$$h_{G,i} = \beta(n) e^{-\frac{|G-i|^2}{2\sigma^2(n)}}$$

Sombrero mejicano: Campos receptivos retina, inhibición lateral



Entrenamiento de un SOM

- Entrenamiento totalmente NO SUPERVISADO (no vemos las salidas deseadas)
- Aprendizaje competitivo, en cada iteración no se ajustan los pesos de toda la red neuronal, solo en principio de la ganadora y algún entorno de ella

Proceso:

1. **Inicialización**
   Hay dos opciones:
   a. Pequeños valores aleatorios con $w_{ji} \epsilon [-0.5, 0.5]$ (peso de neurona j con entrada i)
   b. Eligiendo aleatoriamente $w_j(0) = x_l(l \epsilon [1 \ldots L])$ w y x vectores
   En esta segunda se elige una entrada al azar y se le asigna los valores de ese patrón a los pesos de esa neurona.

2. **Selección del ganador:**
   a. Busco que neurona se parece más al patrón x y la guardo como ganadora.
   $G(x(n)) = argmin_j\{||x(n) - w_j(n)||\}$

3. **Adaptación de los pesos:**
   Si una neurona está en la vecindad de la ganadora, al vector de pesos se le suma el vector de error multiplicado por una

velocidad de aprendizaje, acercando así los pesos al patrón de entrada.

$$\boldsymbol{w}_j(n+1) = \begin{cases} \boldsymbol{w}_j(n) + \eta(n)\left(\boldsymbol{x}(n) - \boldsymbol{w}_j(n)\right) & si\ y_j \in \Lambda_G(n) \\ \boldsymbol{w}_j(n) & si\ y_j \notin \Lambda_G(n) \end{cases}$$

SON TODOS VECTORES LOS DE NEGRITA

PREGUNTAR SI HACE FALTA AGREGAR A LA FÓRMULA H_G,I QUE ES LA DE INHIBICIÓN ACTIVACIÓN

4. **Volver a 2 hasta no observar cambios significativos**

Consideraciones prácticas:

- Generalmente usamos mapas cuadrados.
- Se suele usar la función $h_{G,i}(n)$ uniforme i, pero con un entorno variable decreciente con n. G es la neurona
- La velocidad de aprendizaje $0 < \eta(n) < 1$ también es variable, decreciente con n.

Etapas del entrenamiento

1)Ordenamiento global (o topológico)

$\Lambda_G(n)$ grande (≈ medio mapa)

$\eta(n)$ grande (entre 0.9 y 0.7)

Duración 500 a 1000 épocas

2)Transición

$\Lambda_G(n)$ se reduce linealmente hasta 1

$\eta(n)$ se reduce lineal o exponencialmente hasta 0.1

Duración ≈ 1000 épocas

3)Ajuste fino (o convergencia)

$\Lambda_G(n)$ = 0 solo se actualiza la ganadora

$\eta(n)$ entre 0.1 y 0.01

Duración: hasta convergencia (≈ 3000 épocas)

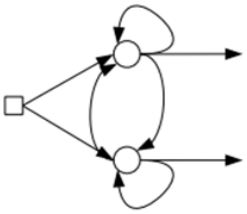
(n) es el indice de iteración, recorre el conjunto de entrenamiento

Si arrancamos con una vecindad chica y tasa de aprendizaje chica cada neurona sólo va a escuchar a sus vecinas inmediatas, y el mapa se puede quedar en un orden global incorrecto, podrian quedar neuronas alejadas donde deberian ser vecinas, entonces para hacer un buen ordenamiento global primero hacemos que el mapa sea grande y luego si hacemos la transición y el ajuste fino

Formación de mapas topológicos

El primer R1 es la cantidad de componentes de entrada, que a su vez es la dimensión de los vectores de pesos de cada neurona. El segundo R1 quiere decir que las neuronas se encuentran ubicadas linealmente (una al lado de la otra). O R2 en un plano

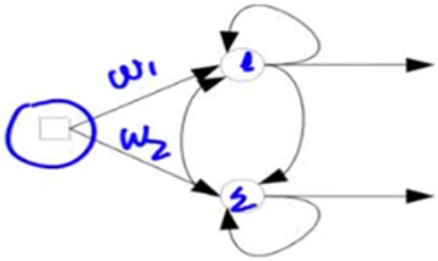
- Ejemplo 1: $\mathbb{R}^1 \rightarrow \mathbb{R}^1$, 2 neuronas



Tengo muchos patrones de un solo número como entrada, supongamos que están cerca de +1 y de -1. Los circulitos representan los pesos que elegimos al azar al comienzo:



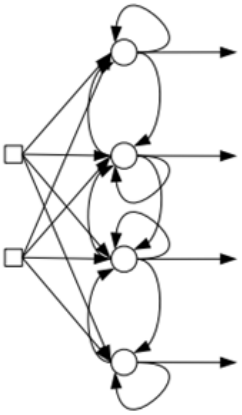
Si entra un patron cercano a +1 gana la neurona 1 porque el error es menor, ajusto los pesos un poco para acercarlo hacia el +1:

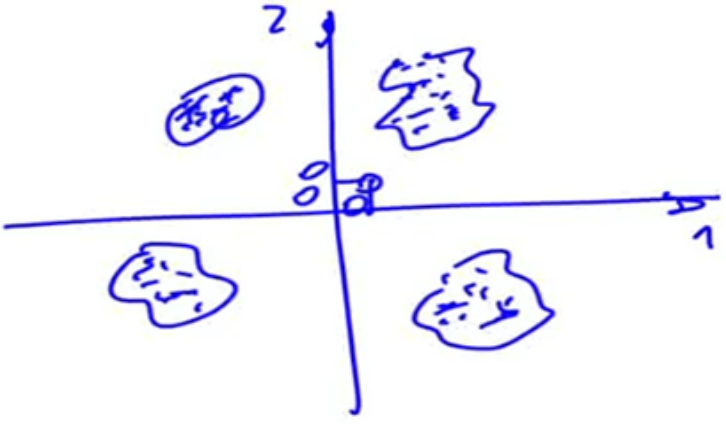




Cuando toque un patrón que haga ganar a la neurona 2, esta se ajustara hacia la izquierda. La neurona 1 terminará en el centro de la distribución de la derecha y la neurona 2 en el centro de la distribución de la izquierda.

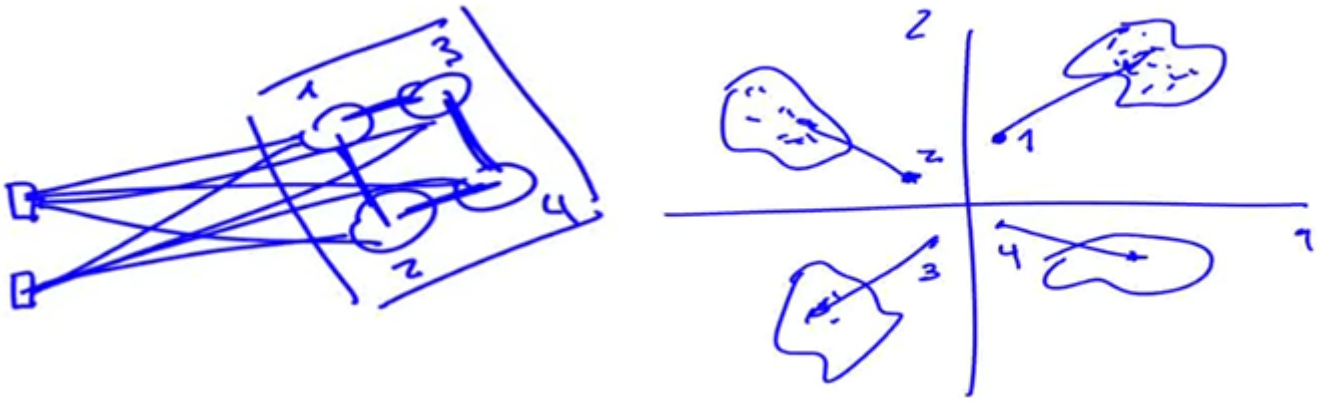
- Ejemplo 2: $\mathbb{R}^2 \rightarrow \mathbb{R}^1$, 4 neuronas





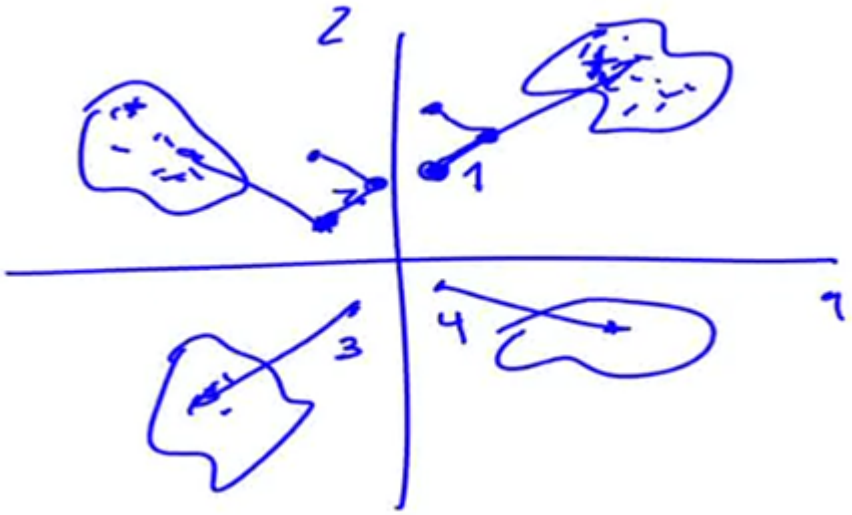
- Ejemplo 3: $\mathbb{R}^2 \rightarrow \mathbb{R}^2$, 4 neuronas

En este caso, las neuronas no están ubicadas linealmente, sino que están ubicadas en un plano. Además, recordemos que las neuronas pueden estar conectadas entre sí dependiendo del tamaño de la vecindad:



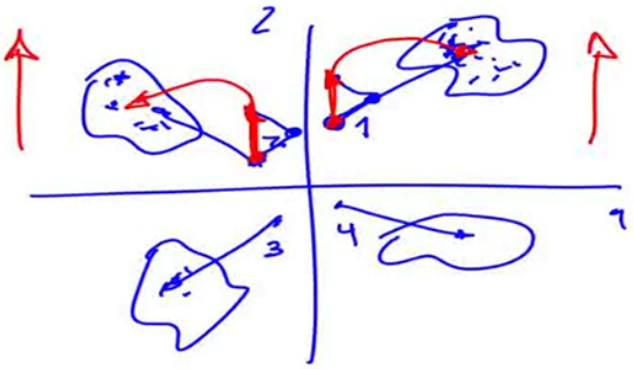
Como vimos antes, si no hubiera vecindad, los pesos se acercan a los centroides

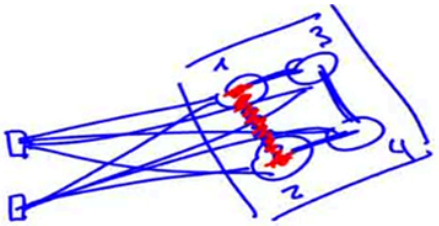
Cuando hay vecindad, si la neurona 1 es vecina de la neurona 2, ambas se mueven juntas al principio:



Esto hace que al comienzo, las neuronas de un entorno se muevan a una misma región del espacio de soluciones sin importar donde hayan comenzado (hacia arriba en imagen) y luego hacia el grupo que les corresponda.
De este modo, las conexiones topológicas entre las neuronas determinan que neuronas que estén cercanas, es decir en una vecindad, también van a representar regiones cercanas en el espacio de solución:

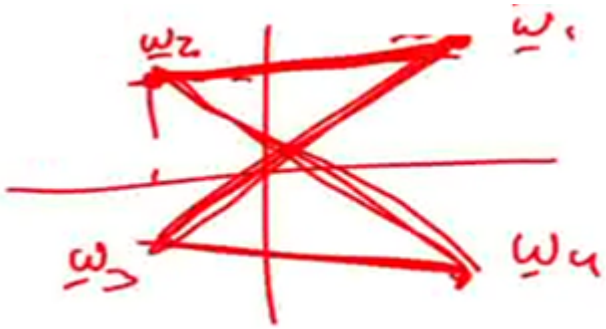
- Ejemplo 3: $\mathbb{R}^2 \rightarrow \mathbb{R}^2$, 4 neuronas





De este modo se va generando el ordenamiento topológico, es decir cómo se logra que neuronas que están cercas en el mapa 2d representan patrones que están cerca en el espacio N dimensional de entrada.

Una forma de graficar esto y que nos permita ver la evolución en el tiempo en R2 es graficar los pesos:



Estos pesos arrancarán cerca del 0,0 y se ira expandiendo en el plano.

- Ejemplo 4: $\mathbb{R}^N \rightarrow \mathbb{R}^2$, M neuronas

El interés principal de los SOM, es que como dije anteriormente, nos permite representar un espacio en dimensiones que no puedo visualizar proyectado en un mapa bidimensional de neuronas donde las neuronas que están cercas representan patrones que también están cerca en el espacio N dimensional.

Un ejemplo de la aplicación de esto es teniendo en cuenta muchas características de los países, agruparlos por similitud.

Cómo utilizar un SOM para clasificar patrones

El SOM solo permite agrupar de manera no supervisada, pero con los siguientes pasos puede servir para clasificar patrones:

1. **Entrenamiento no-supervisado**
2. **Etiquetado de neuronas.**
    a. Para esto ahora si tenemos de que clase es cada patrón, entonces veo cuantas veces se activó la neurona con cada clase y con la clase que más se activó la etiqueto.
3. **Clasificación por mínima distancia**
    a. Ahora cuando entre un patrón de clase desconocida, cuando se active esa neurona, nos dirá que clase es. Para ver cuál es la ganadora veo cual es la mínima distancia entre el patrón de clase desconocida y los pesos.

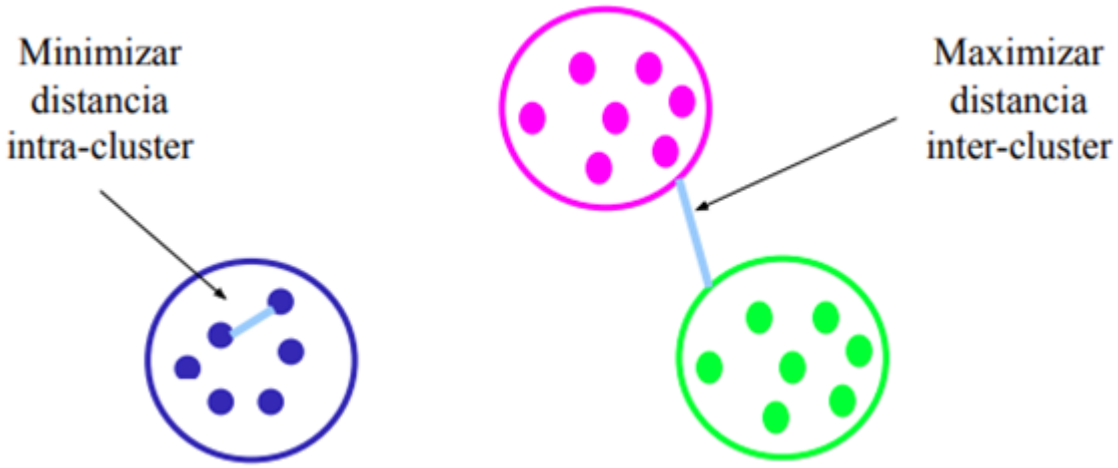
Comparación con otros métodos de agrupamiento: k-medias

Aprendizaje no supervisado = clustering.

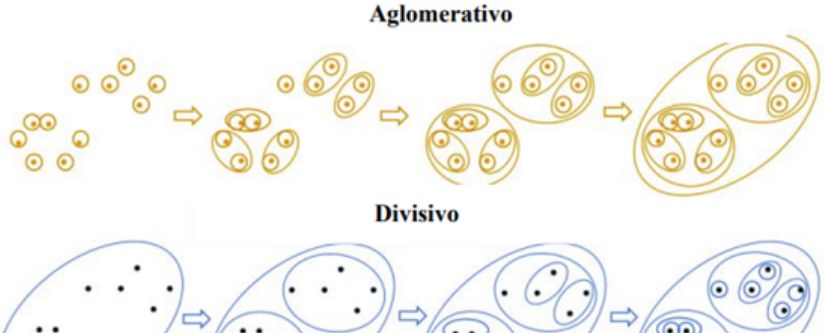
Clustering es agrupar objetos similares basados en alguna noción de similaridad. El resultado es una partición de los datos.

Un **Cluster** es un conjunto de objetos que son similares entre ellos mientras que difieren de los objetos en otros clusters.

Los objetivos de clustering son:



Agrupamiento jerárquico



Aglomerativo: este es un enfoque "de abajo hacia arriba": cada observación comienza en su propio grupo y los pares de grupos se fusionan a medida que uno asciende en la jerarquía.

Divisivo: este es un enfoque "de arriba hacia abajo": todas las observaciones comienzan en un grupo y las divisiones se realizan de forma recursiva a medida que uno desciende en la jerarquía.

K-medias

- Algoritmo muy popular y muy simple.
- Particiona los datos en k clusters.
- Utiliza centroides (prototipos de los clusters).
- K-medias es un algoritmo que agrupa objetos en k grupos basándose en sus características.
- Minimizando la suma de distancias entre cada objeto y el centroide de su grupo o cluster.
- Se suele usar la distancia euclídea.
- Las principales ventajas del método k-medias son que es un método sencillo y rápido.
- Pero… es necesario decidir el valor de k a-priori
- El resultado final depende de la inicialización de los centroides.
- En principio no converge al mínimo global sino a un mínimo local.

Algoritmo
El algoritmo consta de tres pasos:

1. Inicialización: una vez elegido el número de grupos k, se establecen k centroides en el espacio de los datos aleatoriamente.
2. Asignación objetos a los centroides: cada objeto de los datos es asignado a su centroide más cercano.
3. Actualización de centroides: se actualiza la posición del centroide de cada grupo tomando como nuevo centroide la posición del promedio de los objetos pertenecientes a dicho grupo.

Se repiten los pasos 2 y 3 hasta que los centroides no se mueven.

A. k-medias por lotes (batch)
Procesa todos los patrones juntos en cada vuelta, antes de mover los centroides.

1. Inicialización: se arman k conjuntos C□(0), repartiendo los patrones al azar entre esos k grupos
2. Se calculan los centroides: promedio (suma de vectores) de los patrones de cada conjunto
3. Reasignación: cada patrón se mueve al conjunto cuyo centroide tiene más cerca
4. Volver a 2 hasta que no haya más reasignaciones

B. k-medias online

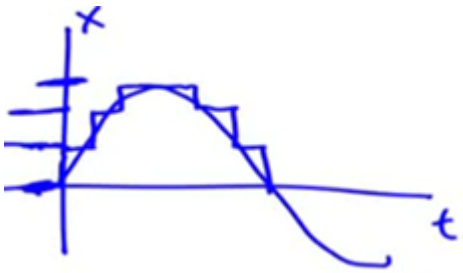
Se ajusta patrón a patrón, uno a uno (no espera a ver todos los patrones antes de mover los centroides)

1. Se eligen k patrones al azar y se usan directamente como centroides iniciales
2. Para cada patrón que entra, se compara su distancia contra todos los centroides. Gana el más cercano
3. Se actualiza solo el centroide ganador, acercandolo un poco hacia el patrón que acaba de entrar
4. Se repite hasta no encontrar mejoras significativas o se acaben los patrones

Cuantización vectorial con aprendizaje (LVQ)

Está inspirado en los mapas autoorganizativos. La idea principal es tratar de cuantizar de manera adaptada a las necesidades del espacio. Cuantizar significa separar en cuantos.
LVQ es un clasificador supervisado.

Conceptos básicos:
Cuantizador escalar: señales cuantizadas en escalones



- **Cuantizador vectorial:** podemos cuantizar espacios no necesariamente de una dimensión. Además, puede cuantizar algunas zonas del espacio con mayor detalle y otras con menos.
    - **Centroides o prototipos**
      Los prototipos son el centro de un grupo de patrones. Cuando hablaba de neuronas, una neurona tenía un vector de pesos que era un centroide o un prototipo.
    - **Diccionario o code-book**
      Un conjunto de prototipos hace a un diccionario de prototipos.
    - **El proceso de cuantización:** de vectores a números enteros

La idea es que yo pueda recibir unos vectores en RN y guardar en un archivo comprimido el índice del prototipo que es mas parecido.

Descompongo así por ejemplo una imagen en muchos prototipos o zonas y guardo el índice del prototipo al que más se le parece. Por ejemplo, el prototipo guardado puede ser una curva que es una especie de promedio de varias curvas.

Luego cuando quiero descomprimir voy a poner en esa zona de la imagen el prototipo. Obvio voy a tener un error.

Ideas de como entrenarlo:

- **Algoritmo k-medias etiquetado para clasificación.**
  Los prototipos son los centroides etiquetados de k-medias.
- **SOM etiquetado como clasificador**
  Los prototipos o cuantos son las neuronas etiquetadas del SOM.
- **Algoritmo supervisado LVQ**
  A diferencia de k medias y SOM que realizan el etiquetado y la supervisión aparecen luego del entrenamiento, LVQ nos permite clasificar durante el entrenamiento, es supervisado desde el entrenamiento.

Algoritmo LVQ1

1. **Inicialización aleatoria**
   Asignamos a los cuantos/prototipos valores pequeños al azar o igual a algún patrón del entrenamiento.

$$\boldsymbol{m}_i(0) \in [-0.5, 0.5]; o\ \boldsymbol{m}_i(0) = rnd(\boldsymbol{x});\ \ \forall i$$

2. **Selección**
   Dado un patrón, busca cual es el centroide o prototipo más cercano a ese patrón:

$$c(n) = \arg min_i\{\|\boldsymbol{x}(n) - \boldsymbol{m}_i(n)\|\}$$

3. **Adaptación:**

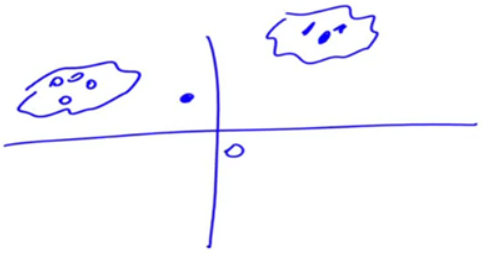
$$\boldsymbol{m}_c(n+1) = \boldsymbol{m}_c(n) + s(c,d,n)\alpha[\boldsymbol{x}(n) - \boldsymbol{m}_c(n)]$$

$$s(c,d,n) = \begin{cases} +1 & si\ \mathcal{C}\big(\boldsymbol{m}_c(n)\big) = d(n) \\ -1 & si\ \mathcal{C}\big(\boldsymbol{m}_c(n)\big) \neq d(n) \end{cases}$$

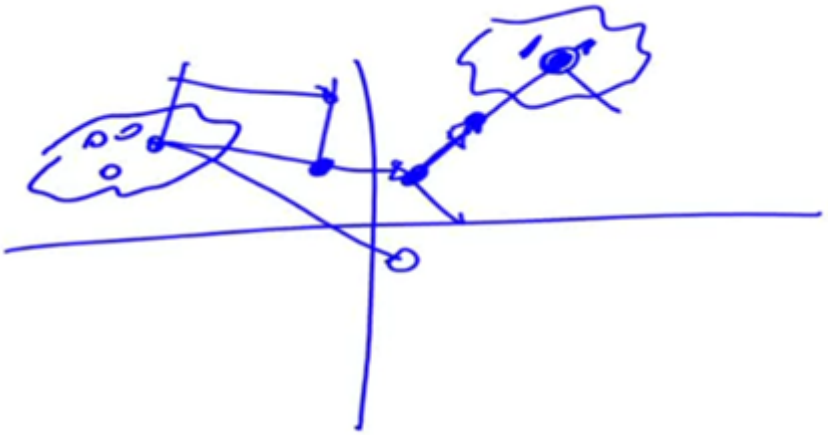
$s(c,d,n)$ es un factor que dice si acierto a la clase o no. Es decir, si la clase del prototipo ganador es igual a la clase del patrón . Si lo acierta acerca el prototipo al patrón que acaba de entrar, si no, lo aleja. es la velocidad de aprendizaje.

4. **Volver a 2 hasta satisfacer error de clasificación.**

Acá no hay vecindad como en el SOM.



Si llega un patrón de los de la izquierda, el de menor distancia es el circulo pintado, pero como no es de la misma clase se mide el vector de error y se aleja.



Entonces cuando es incorrecta la aleja y cuando es correcta la acerca.

Luego para clasificar entradas desconocidas se medirá la distancia mínima entre el patrón y los prototipos. La clase del prototipo ganador será la clase del patrón.

Observaciones

- No hay arquitectura neuronal
- Se puede ver al cuantizador como SOM lineal, sin entorno y supervisado
- Velocidad de aprendizaje
    - ¿Existe un óptimo para cada centroide?
    - ¿Se debe considerar un optimo para cada instante de tiempo?

Si desarrollamos:

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\,[\mathbf{x}(n) - \mathbf{m}_c(n)]$$

$$\mathbf{m}_c(n+1) = \mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n) - s(n)\alpha(n)\mathbf{m}_c(n)$$

$$\mathbf{m}_c(n+1) = [1 - s(n)\alpha(n)]\,\mathbf{m}_c(n) + s(n)\alpha(n)\mathbf{x}(n)$$

$$\mathbf{m}_c(n+1) =$$

$$= [1 - s(n)\alpha(n)]$$
$$\{\mathbf{m}_c(n-1) + s(n-1)\alpha(n-1)\,[\mathbf{x}(n-1) - \mathbf{m}_c(n-1)]\}$$
$$+s(n)\alpha(n)\mathbf{x}(n)$$

$\mathbf{x}(n-1)$ es afectado dos veces por α

Siendo $\alpha < 1$ la influencia de los primeros patrones de entrenamiento siempre será menor que la de los últimos. Esto se da porque el último patrón va a estar multiplicado por un solo α el anterior por 2, el anterior por 3 y así sucesivamente.

Lo que genera esto es que se olvide rápido: las cosas que primero le mostramos van a tener muy poca influencia en la posición del centroide.

Si queremos que α afecte por igual a todos los patrones debemos hacerlo decrecer con el tiempo.

Se debe cumplir que:
$\alpha_c(n) = [1 - s(n)\alpha_c(n)]\alpha_c(n-1)$

Es decir, que el α en dos iteraciones consecutivas valga l omismo. A la izquierda tengo el $\alpha(n)$ y todo a la derecha es $\alpha(n-1)$
Obtengo asi que el α en la iteracion n debe valer:

$$\alpha_c(n) = \frac{\alpha_c(n-1)}{1 + s(n)\alpha_c(n-1)},\ \ no\ sobrepasar\ \alpha > 1$$

Puedo arrancar con $\alpha = 0.9$ y el siguiente calcularlo con esa formulita. Haciendo esto logro que todos los patrones a lo largo del entrenamiento tienen la misma influencia, el mismo peso en el valor final de los centroides.

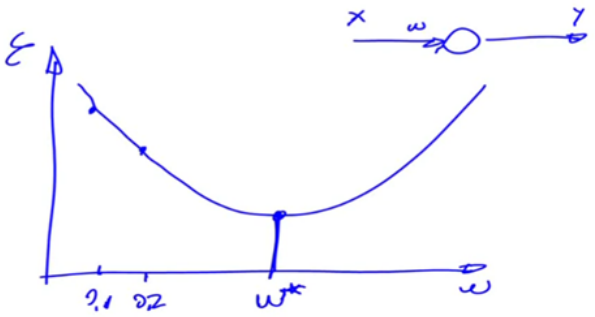
UNIDAD 4

Capacidad de Generalización

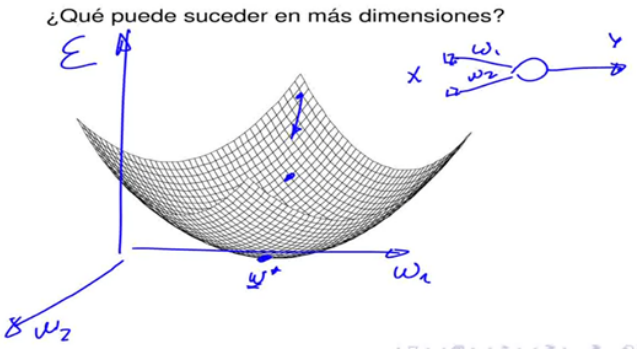
Es la capacidad que tienen los sistemas para funcionar bien en aquellos casos que no vieron durante el entrenamiento

Superficies de error

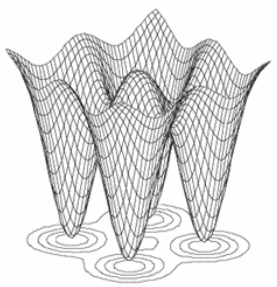
Supongamos que hay un único parámetro para ajustar...



¿Qué puede suceder en más dimensiones?

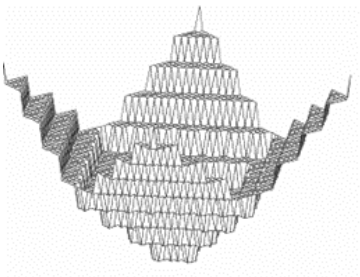


Con muchos mínimos:



Un método para no caer en un mínimo local puede ser inicializar con los pesos al azar muchas veces, para que arranque en distintos puntos de la superficie y me quedo con el mejor mínimo.

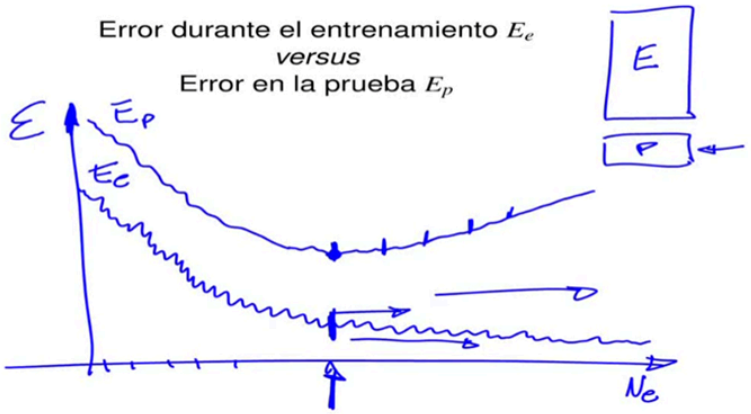
En este caso el método del gradiente se puede quedar en una meseta.



Sobre entrenamiento

Es cuando un modelo se ajusta mucho a los datos de entrenamiento y pierde capacidad de ajustarse a datos que nunca vio.
Durante el entrenamiento siempre se va a mejorar el error a medida que pasan las epocas, pero en la prueba, puede ser que llegue un punto en el que el error empiece a subir, ese punto es donde el algoritmo aprendió detalles de los datos de entrenamiento como el

ruido propio de la medición, etc., que no es propio del problema que queremos resolver. Esta sobre ajustando o perdiendo capacidad de generalización:



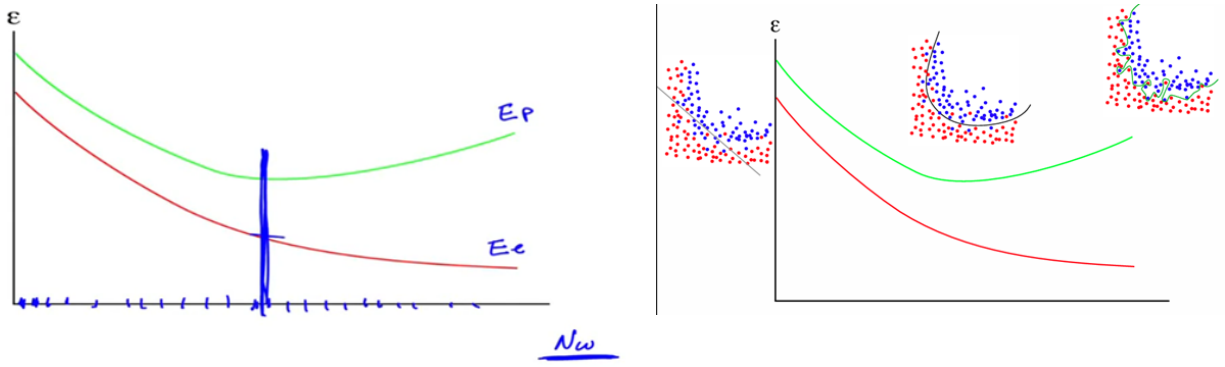
Ne → número de épocas.

Ee → error entrenamiento.

Ep → error prueba.

Early stopping es el método de cortar el entrenamiento en ese punto.

Lo mismo puede suceder agregando complejidad al sistema, esto puede ser agregando la cantidad de pesos, es decir la cantidad de neuronas:

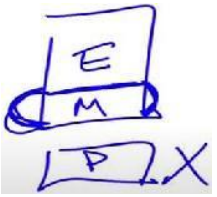


¿Qué puedo hacer para EVITAR EL SOBRE-ENTRENAMIENTO?

→ Cuando vemos que el error de prueba sube, cortamos el entrenamiento.

→¿Pero no estamos ajustando según los datos de prueba entonces?
→ sí, y en realidad estos no deberán usarse para nada

=> tenemos que tener una estrategia para medir sólo con los de entrenamiento:



easy → nos guardamos una porción de los datos de entrenamiento y le llamamos un subconjunto de monitoreo.

POR EJ: CANTIDAD DE DATOS = 100%
TEST=80%
PRUEBA=80%

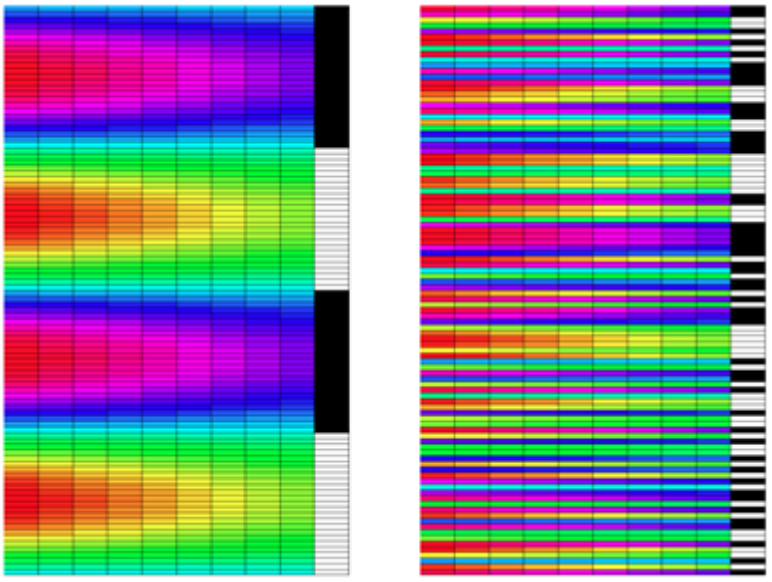
Y despues divido el test nuevamente en TEST = Entrenamiento 80%
Monitoreo 20%

Estimación de la capacidad de generalización

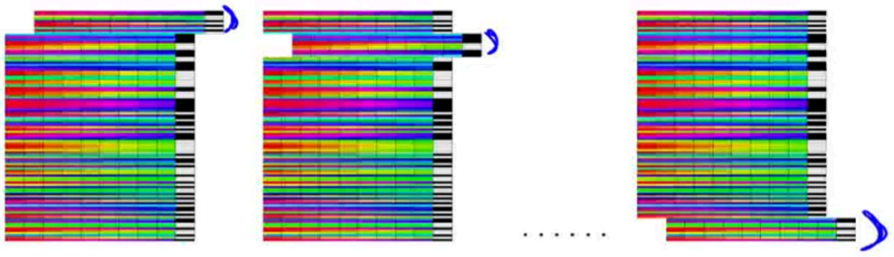
¿Cómo podemos ESTIMAR la capacidad de generalización?
(que tan bien va a funcionar con datos que no son estrictamente los que tengo para entrenarlo y esa es la verdadera tasa de error acierto que quiero conocer)

Validación Cruzada

El dataset generalmente viene ordenado por clases, por lo que la desordeno para no tomar de validación todos de la misma clase:



Se realizan particiones de prueba y se entrena con dichos subsets:



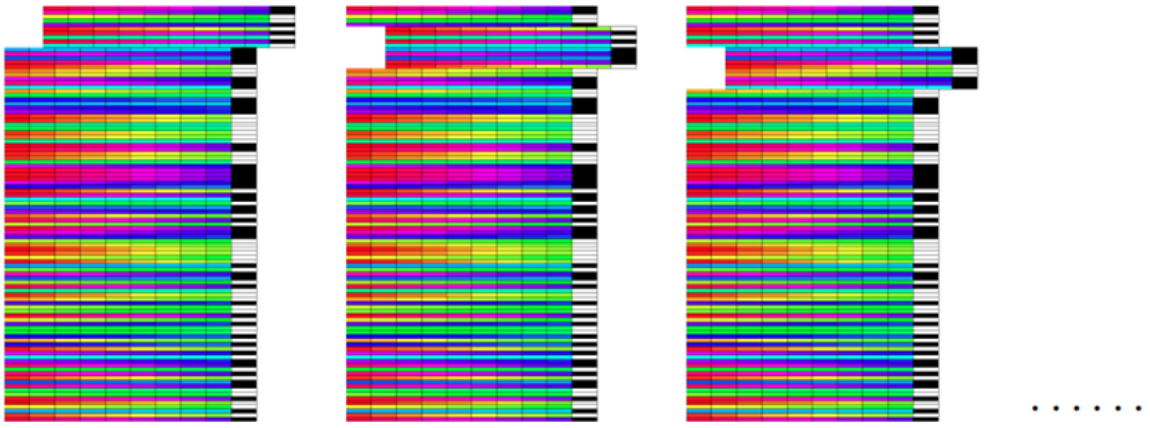
e_{P_1} e_{P_2} --- e_{P_i} - - - e_{P_N}

$$\mathcal{E} = \frac{1}{N} \sum_{i=1}^{N} e_{P_i}$$

Además de la media del error puedo medir la varianza para ver si las particiones son buenas o si los distintos errores son muy distintos.

Este método se llama leave-k-out, dice cuántas se dejan afuera. k-fold es cuantas particiones hay, en este caso N particiones.

Otro método es el leave-1-out
Otra manera es no realizar particiones excluyentes y realizar solapamientos:



Medidas de desempeño

Hablamos de medir el error, esta parte define como medimos ese error.

Para clasificador binario
Matriz de confusión

Matriz de confusión

		Predicciones $\oplus$	Predicciones $\ominus$	
Clases correctas	$\oplus$	$t_\oplus$	$f_\ominus$	$N_\oplus = t_\oplus + f_\ominus$
	$\ominus$	$f_\oplus$	$t_\ominus$	$N_\ominus = f_\oplus + t_\ominus$
				$N = N_\oplus + N_\ominus$

Verdaderos positivos: eran positivos y acerto. Falsos negativos: eran positivos y dijo que eran negativos.
En estos lugares se ponen la cantsuperidad de casos de dicho estilo.
N+ es la cantidad de casos positivos que había, N- la cantidad de casos negativos, N la cantidad de casos.
En base a esta matriz se definen los índices de medidas:

Sensibilidad y especificidad

$$s^+ = \frac{t_\oplus}{t_\oplus + f_\ominus} = \frac{t_\oplus}{N_\oplus} \rightarrow \left\{ \begin{array}{l} \textbf{sensitivity} \\ \text{true positive rate} \\ \text{hit rate} \\ \text{recall} \\ \text{power} \end{array} \right.$$

$$s^- = \frac{t_\ominus}{t_\ominus + f_\oplus} = \frac{t_\ominus}{N_\ominus} \rightarrow \left\{ \begin{array}{l} \textbf{specificity} \\ \text{true negative rate} \end{array} \right.$$

Precisión

$$p = \frac{t_\oplus}{t_\oplus + f_\oplus} \rightarrow \left\{ \begin{array}{l} \textbf{precision} \\ \text{positive predictive value} \end{array} \right.$$

$$n = \frac{t_\ominus}{t_\ominus + f_\ominus} \rightarrow \left\{ \text{negative predictive value} \right.$$

Exactitud y otras medidas compuestas

$$a = \frac{t_\oplus + t_\ominus}{N} \longrightarrow \{ \textbf{accuracy}$$

$$F_1 = \frac{2t_\oplus}{2t_\oplus + f_\oplus + f_\ominus} = 2\frac{s^+ p}{s^+ + p} \longrightarrow \begin{cases} F_1 \textbf{ score} \\ \text{F-score/measure} \\ \text{harmonic mean} \end{cases}$$

$$G = \sqrt{s^+ p} \longrightarrow \begin{cases} \textbf{G-measure} \\ \text{geometric mean} \end{cases}$$

Errores relativos

Dos clasificadores tienen

$a_1 = 50\,\% \longrightarrow 50$
$a_2 = 51\,\% \longrightarrow 49$

$\frac{50-49}{50}$

$\frac{1}{50}$

mientras que otros tienen

$a_3 = 98\,\% \longrightarrow 2$
$a_4 = 99\,\% \longrightarrow 1$

Supongamos que $e = 1 - a_r$
el error relativo a la referencia e_r es

$$\delta_e = \frac{e_r - e}{e_r}$$

$\frac{2-1}{2} = \frac{1}{2}$

e_r sería el clasificador de referencia, por ej a1 o a3.

Parece que en los dos casos el clasificador 2 y 4 mejoran la misma cantidad respecto al 1 y 3, pero no. Para eso sirve el error relativo.

Aprendizaje No Neuronal (kNN, Árboles, Naive Bayes, SVM)

kNN - k-Nearest Neighbors

- Método NO paramétrico
- En realidad no aprende
- Vuelve a buscar los más cercanos con cada nuevo caso a clasificar (basado en instancias).

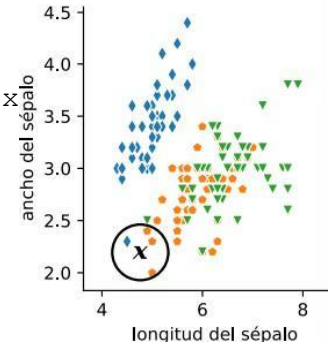
Clasificador kNN

Supongamos que tenemos:

- N_k puntos de la clase C_k
- N puntos en total $(\sum_k N_k = N)$

Para un nuevo x, podemos imaginar una esfera centrada en x
→ Supongamos que esta esfera:

- tiene un volumen V
- contiene K puntos en total



- de los cuales Kk son de la clase Ck

La densidad de la clase k en la esfera sería → $p(\mathbf{x}|C_k) = \frac{K_k}{N_k V}$ Y si no condicionamos a la clase tendríamos → $p(\mathbf{x}) = \frac{K}{NV}$

Lo que podríamos esperar para cada clase sería → $p(C_k) = \frac{N_k}{N}$

→ Si con esto usamos el teorema de Bayes

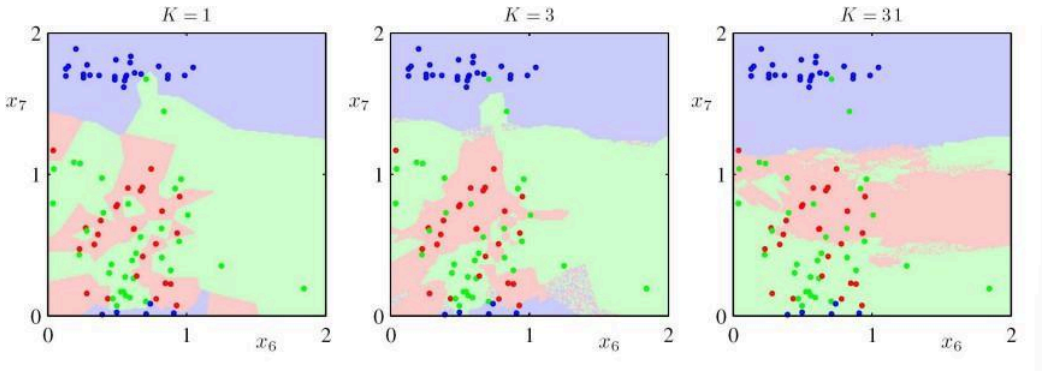
$$p(C_k|\mathbf{x}) = \frac{p(\mathbf{x}|C_k)p(C_k)}{p(\mathbf{x})} = \frac{\frac{K_k}{N_k V}\frac{N_k}{N}}{\frac{K}{NV}} = \frac{K_k}{K}$$

= probabilidad de que el patrón dado que sea de la clase, la probabilidad del patrón / probabilidad de clase

Ventajas:
✔ Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
✔ No hay parámetros entrenables
✔ Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento
✔ Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

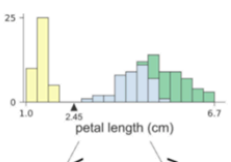
Desventajas:
✖ Hay que determinar el valor óptimo de k.
✖ No es tan claro que distancia usar en cada caso.
✖ Para cada dato de prueba se requiere calcular la distancia a todos los datos de entrenamiento.
✖ Sensible a outliers y clases desbalanceadas.

A más vecinos, más detalle:



Árboles de decisión

Son clasificadores que utilizan reglas IF-THEN para separar los datos en función de sus atributos.



1. Separa si el largo del petalo es menor o mayor a 2.45

2. Menor a 2.45= setosa mayor= versicolor+virginica

2. separa nuevamente por el largo en 4.75

3. luego separa por el ancho.

¿Como elegimos por qué dimensión (nodo) dividir primero? Con las siguientes métricas:

Impureza de clasificación:

$$\mathcal{I}_n^C = 1 - \max_i \mathsf{P}(\mathbf{x} \in w_i | n)$$

Impureza de Gini:

$$\begin{aligned} \mathcal{I}_n^G &= \sum_{i \neq j} \mathsf{P}(\mathbf{x} \in w_i | n) \mathsf{P}(\mathbf{x} \in w_j | n) \\ &= 1 - \sum_i \mathsf{P}^2(\mathbf{x} \in w_i | n) \end{aligned}$$

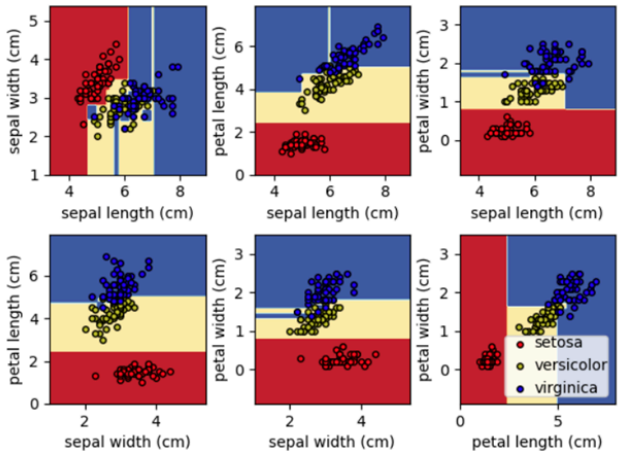
Impureza de Entropía:

$$\mathcal{I}_n^H = -\sum_i \mathsf{P}(\mathbf{x} \in w_i | n) \log \mathsf{P}(\mathbf{x} \in w_i | n)$$

Ganancia de informacion: G = I_p - \sum_h (Nh/N I_h)

Árboles de decisión: fronteras de decisión

Si graficamos las fronteras por cada par de atributos podemos ver las regiones que asigna a cada clase:



Ventajas y desventajas

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.

- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.
- No requieren grandes cantidades de datos para entrenar.
- Bastante robustos a los outliers.

Desventajas

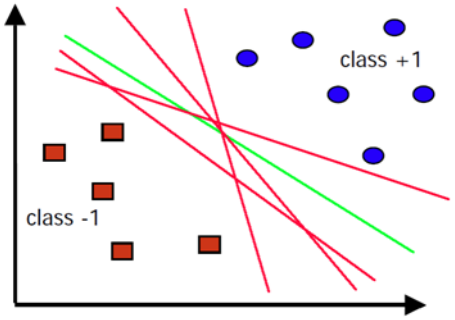
- No son robustos a ruido (pequeños cambios) en los datos.
- Fronteras de decisión ortogonales a las dimensiones de entrada.
- No escalan bien con la complejidad de los problemas a resolver.
- Se sobre ajustan con facilidad

Máquinas con vectores de soporte (SVM)

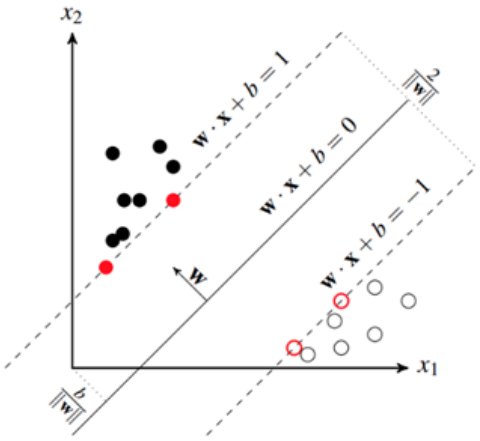
SVM es un algoritmo de aprendizaje supervisado utilizado principalmente para clasificación.

Su objetivo es encontrar un **hiperplano óptimo** que separe dos clases de datos con el **mayor margen posible**.

Margen: cuál sería el mejor clasificador?



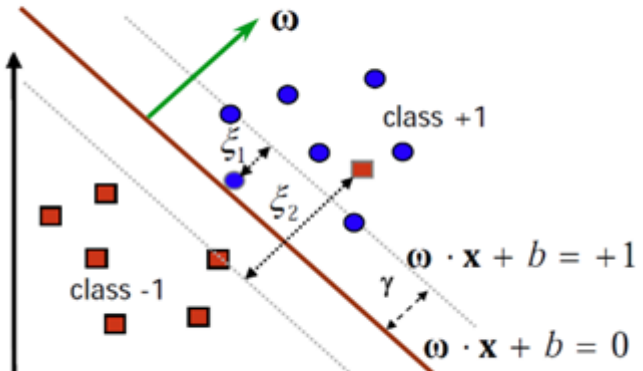
SVM: maximizando el margen



Vectores de soporte

Los puntos en rojo son los **vectores de soporte**, es decir, los puntos más cercanos al hiperplano separador que definen su posición.

SVM: márgenes blandos

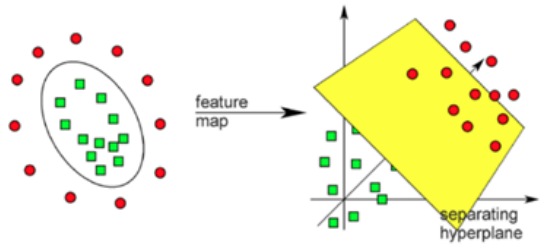


Cuando las clases no son separables linealmente se utilizan márgenes blandos para permitir algunos errores.

El truco del kernel

El **truco del kernel** (kernel trick) es una técnica que permite a las **Máquinas de Vectores de Soporte (SVMs)** manejar problemas donde los datos **no son linealmente separables** en su espacio original. En lugar de trabajar en el espacio original de los datos, el truco del kernel proyecta los datos a un **espacio de mayor dimensión** donde sí pueden ser separados linealmente

Supón que tienes datos en **2D** que no pueden separarse con una línea recta. Aplicando un **kernel**, los datos se transforman en **3D**, donde ahora sí se pueden separar con un plano.



$$\phi : \mathbb{R}^2 \to \mathbb{R}^3$$

$$\phi : (x_1, x_2) \mapsto (z_1, z_2, z3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

Ventajas y desventajas

Ventajas:

Rápida convergencia en entrenamiento.
Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
Funciona bastante bien en altas dimensiones de entrada.
Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
Suele proveer buena capacidad de generalización.

Desventajas:

El desempeño es dependiente del kernel que se elija (y sus hiperparametros).
Tiene problemas con grandes conjuntos de datos.
No maneja bien el solapamiento de clases.
No provee buena interpretabilidad de las decisiones.
No es directa la extensión a problemas multiclase.

Ensambles de Clasificadores (Bagging, Stacking, Boosting, AdaBoost, Gradient Boosting)

Ensambles de clasificadores

Bagging

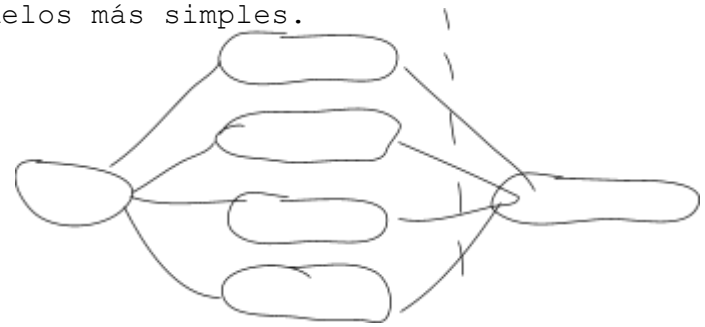
Entrenar varios modelos simples…
haciendo bootstrap sobre el conjunto de datos

1. Entrenar B modelos separados, cada uno para predecir yb(x).
2. La predicción del ensamble estará dada por simple votación de las salidas de cada modelo yb(x) (o promedio en caso de regresión).

$$yens(x) = \sum_{b=1}^{B} yb(x)$$

Stacking

Entrenas varios modelos simples y luego un meta modelo recibe como entrada las salidas de los modelos más simples.



Se entrenan por separado los modelos simples del metamodelo.

Boosting

Ideas clave:

- A diferencia del bagging, en boosting modificamos la distribución de los datos de entrenamiento antes de hacer un nuevo muestreo con bootstrap.
- Buscamos que la nueva muestra sea más informativa para el nuevo modelo a entrenar.

Boosting por muestreo 3

Dado un conjunto de entrenamiento S de n patrones, para un clasificador binario:

1. Se entrena el clasificador C1 usando la muestra de bootstrap S1 de n' < n patrones (sin reposición).
2. Se entrena el clasificador C2 en un conjunto de entrenamiento S2 en el que la mitad de sus patrones hayan sido clasificados correctamente por C1.
3. Se entrena el clasificador C3 con los patrones en los que la predicción de C1 y C2 difieren.

Adaboost

- Boosting adaptativo. La idea clave: prestar más atención a los patrones difíciles.
- Cada patrón de entrenamiento tiene asociado un peso que determina la probabilidad con la que va a ser elegido en el boosting.
- Si el patrón se clasifica bien entonces la probabilidad de ser elegido nuevamente tiene que bajar, y a la inversa, aumentar si es más clasificado.
- En un test se usa un voto pesado de acuerdo al desempeño previo de cada clasificador del ensamble. (cuanto menos error haya tenido, mas vale su voto)

Convergencia de AdaBoost



A medida que agrego más clasificadores débiles en el boosting, cada clasificador débil nuevo comete más error porque el algoritmo va aumentando los pesos de los patrones difíciles, así que cada clasificador ve un problema más complicado que el anterior.

Sin embargo, el ensamble completo mejora cada vez más: el error en entrenamiento baja continuamente y el error en test también disminuye, porque cada clasificador corrige los errores del anterior.

Ventajas y desventajas de los ensambles

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalizacion).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de multiples clasificadores y fuentes/tipos de datos.

- Hacen un mejor aprovechamiento de los datos cuando son escasos.
- Tratan naturalmente los problemas multiclase.
- Estables y robustos.

Desventajas:

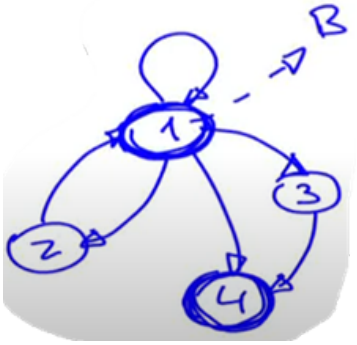
- El entrenamiento puede ser costoso.
- No son tan fáciles de interpretar.
- Los resultados pueden variar mucho dependiendo de la elección de los modelos débiles, su diversidad y la de los datos.

UNIDAD 5

Autómatas Celulares y Agentes

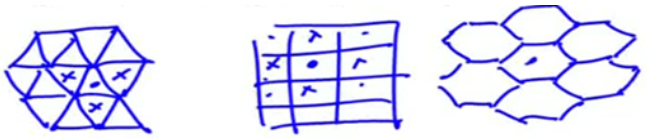
AUTÓMATAS DE ESTADOS FINITOS: $A =< X, Y, E, D >$
Es una estructura que tiene:
X = entradas, Y = Salidas, E = conjunto de estados, D = Reglas que permiten al autómata ir cambiando de estados

Un autómata se representa mediante un grafo: se pueden visualizar los estados, las reglas de transición y si hay salidas. En algunos casos también se pueden considerar entradas.

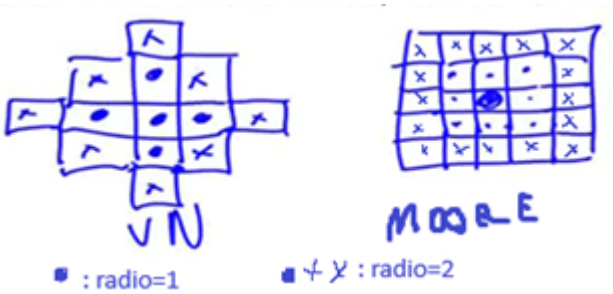


AUTÓMATAS CELULARES: $R =< A, T, C >$
Definición: Son otra estructura compuesta por
A = autómatas, T = topología, C = cómo se conectan entre sí
Vendría a ser un conjunto de autómatas que se vinculan entre sí.

- **Topologías (T)**
  - Triangular
  - Rectangular
  - Hexagonal
  - etc



- **Acoplamiento (C):** Dentro de una topología como se conectan entre sí
  - Tipos y tamaños de vecindad:
    - Von neuman: solo los que comparten un lado
    - Moore: diagonales



Ejemplos de autómatas celulares: Juego de la vida de Conway: un juego que para un organismo, dependiendo de los organismos del entorno se define si vive o muere. con esto se puede simular el crecimiento de plantas, bacterias, poblaciones, tejidos biológicos: cardiaco, nervioso. Fluidos.

Agentes

Un agente es un sistema que:

- Está situado en un ambiente
- Es capaz de realizar acciones automáticas
- Tiene objetivos de diseño

Un agente es todo aquello que:

- Percibe su ambiente mediante sensores
- Responde o actúa sobre el ambiente mediante efectores

Todo agente debe poseer autonomía, es decir, capacidad de:

- Aprender de la experiencia
- Modificar comportamiento en tiempo de ejecución

Un agente inteligente debe:

- Ser proactivo: realizar acciones sin requerimiento específico.
- Ser reactivo: reacciona
- Poseer habilidad social: interactuar con otros agentes

Un agente racional es aquel que

- Realiza acciones correctas

Un agente racional ideal debe ser capaz de:

- Percibir
- Conocer
- Decidir
- Actuar

Sistemas multi-agente

Son cooperativos:

- Por interacción
- Por contratos
- Por negociación

Un agente posee racionalidad social:

- Si puede realizar acciones que generan un beneficio a todos
- Si ese beneficio es más grande que las perdidas

$$\text{utilidad esperada} = f(\text{utilidad individual}) + f(\text{utilidad social})$$

Inteligencia colectiva: conceptos básicos

Denominaciones y relaciones:

- Computación evolutiva: basados en la teoría de Darwin
- Inteligencia de colonias
- Inteligencia de enjambres
- Inteligencia colaborativa
- Inteligencia social

Características generales:

- Auto-organización (relación con SOM...). Es la capacidad de armar una topología o un mapa que tiene que ver con la estructura de los datos en un espacio n dimensional. No la definimos nosotros sino que se va ordenando automáticamente.
- Estigmergia: "colaboración a través del medio físico". Ej rasgo de hormonas en colonia de hormigas.
- Comportamiento emergente: inteligencia distribuida, robustez. Es un comportamiento que no es definido de antemano, sino que emerge a partir de la colaboración de individuos sencillos
- Fuerte interacción local. Fuerte interacción con los que están cerca.
- Organización social altamente estructurada. Se empiezan a formar roles o jerarquías.
- Colaboración versus competencia
- Componentes estocásticas. Son los valores que se determinan al azar.
- Bio-inspiración

Ejemplos de bio-inspiración:

- Bandadas de pájaros
- Colonias de hormigas
- Paneles/enjambres de abejas
- Cardúmenes de peces
- Rebaños de ovejas o cabras
- Manadas de predadores

Elementos individuales:

- "Boids": partículas, objetos, elementos...
- Autómatas: redes de autómatas celulares
- Agentes: en estructuras de multi-agentes
- Neuronas...? como con el som.

Algoritmos(con tilde los que se ven en la materia):

✔ Algoritmos evolutivos
✔ Colonias de hormigas
✔ Enjambre de partículas

Principales aplicaciones:

- Optimización: aproximación de funciones, entrenamiento, estimación, identificación, planificación,...
- Búsqueda, ruteo
- Agrupamiento no supervisado (clustering), clasificación

Métodos evolutivos

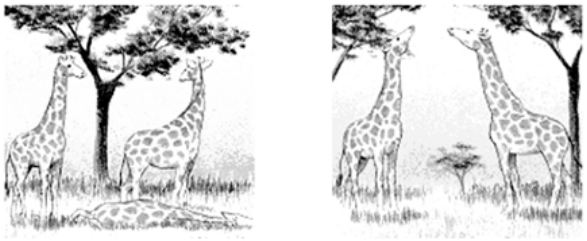
Creacionismo: las especies de la naturaleza fueron creadas como tales
Evolucionismo: hubo cambios a lo largo de los años
Lamarck: creía que los caracteres adquiridos en vida iban a ser heredados a la siguiente generación.

Por ejemplo, una generación nueva iba a tener el cuello más largo que la anterior.

Ahora sabemos que los caracteres adquiridos en vida no se heredan, sino que ya están codificados en el material genético.

Teoría de la evolución de Darwin: Variación y selección natural

Variación y selección natural: si hay variabilidad en la longitud del cuello de las jirafas, las de cuello corto tendrán menos probabilidades de sobrevivir y procrear. Así...



...en la próxima generación habrá menos jirafas de cuello corto.

Inspirándonos en esto, surge la evolución como un algoritmo:

La evolución como un algoritmo

Inspirado de

✔ El creacionismo y las ideas de Lamarck
✔ Darwin versus Lamarck

- Poblaciones versus individuos
- Mejores versus "adaptados": busca la mejor solución a las condiciones actuales, no quiere decir que sea el mejor global.
- Aleatoriedad en la selección natural
- Diversidad y operadores de variación en la población

La evolución como un algoritmo

```
Inicializar(Población)
MejorAptitud ← Evaluar(Población)
mientras MejorAptitud < AptitudRequerida
    Progenitores ← SelecciónNatural(Población)
    Población ← ReproducciónVariación(Progenitores)
    MejorAptitud ← Evaluar(Población)
fin
```

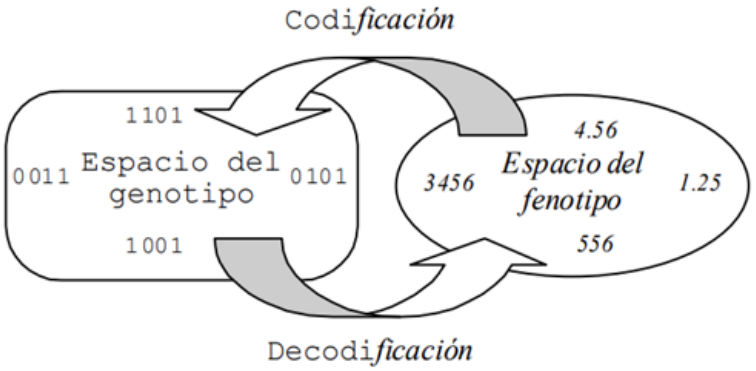
la población son potenciales soluciones al azar.

Elementos de un algoritmo evolutivo

- Representación de los individuos
- Función de aptitud (o fitness)
- Mecanismo de selección
- Operadores de variación
- Reproducción y reemplazo generacional

Variantes de la computación evolutiva

Basándonos en cómo se **representan los individuos**, el algoritmo/programación/estrategia puede ser:

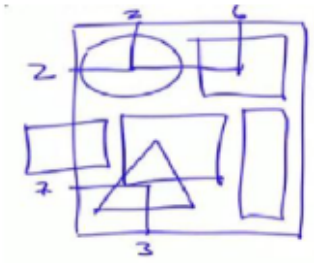


- Genético: Representación BINARIA o "genotípica"
    - Muchos genes con pocos alelos (2: 1 y 0): convergencia asegurada por el teorema de esquemas.

    - Epítasis: un gen incorrecto invalida a todo el cromosoma (un cambio en un bit puede generar un valor muy distinto o invalido en el fenotipo)
    - Representación lejana al dominio del problema (estoy transformando del dominio del problema al genotipo)
    - Gran cantidad de soluciones invalidas en la población (no se diseña bien la representación de los individuos la población va a tener muchos individuos inválidos)
- Evolutivo (o estrategias de evolución): representación REAL o "fenotípica"
    - Pocos genes con muchos alelos: representación fenotípica.
    - Convergencia muy dependiente de los operadores
    - Necesidad de redefinición de operadores no "biológicos" (redefinir el operador de cruza, mutación porque mutar ya no es cambiar un 0 por un 1 si es en un número real)

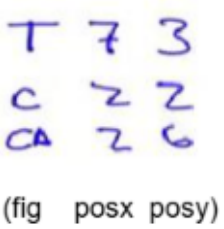
EJEMPLO DE REPRESENTACIÓN DE GENOTIPO A FENOTIPO

Solución: (pos. figuras)



Solución codificada:

(fenotipo)



(fig posx posy)

Cromosoma binario: ***(genotipo)***

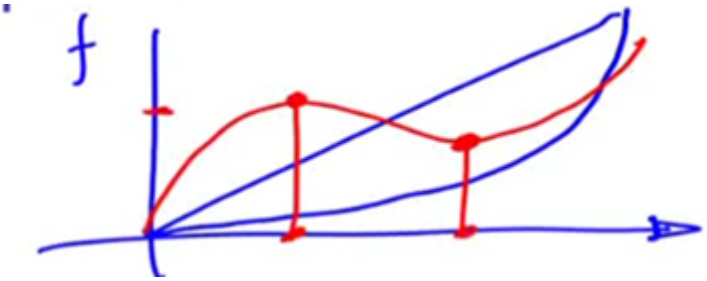
(3 figuras, 10 bit en total por cada una) [fig posx posy]

[00 0000 0000][01 0000 0000]...[10 0000 0000]

Función de aptitud

Como medir que tan bueno es un individuo para resolver el problema. Nos sirve para elegir a los padres de la siguiente generación.

- Características generales:
    - Monotonicidad: creciente
      Rojo es erróneo, a mayor bondad del individuo, debe mayor fitness.



    - Precisión: debe diferenciar individuos que sean apenas parecidos

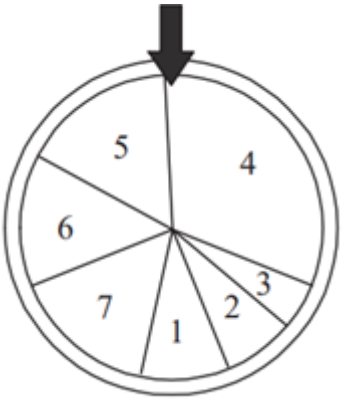
- Suavidad regulable: debo poder variar la suavidad (ej en la sigmoidea variando el alpha)
- Penalización de complejidad: ante una solución simple y una compleja debe elegir la simple.

- Algunos ejemplos típicos:
  - Promedios de error: cuadrados medios, desviación media absoluta, error relativo medio,...
  - Estadísticas: estimación de la varianza, validación cruzada, verosimilitud, predicción de error,...
  - Medidas de información: criterio de Akaike, criterio de información Bayesiano, descriptor de mínima longitud, información mutua, minimización del riesgo empírico
  - Otras: correlaciones, distancias,...

Estrategias de selección

(de padres para la próxima generación)

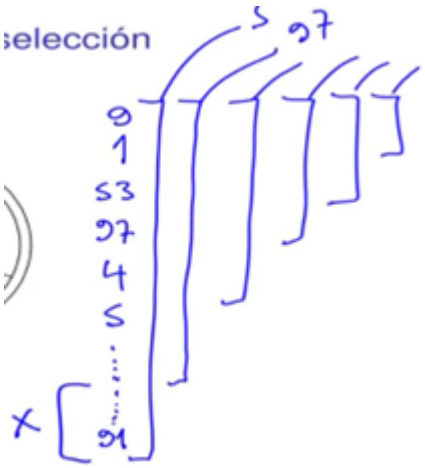
- **Rueda de ruleta**
  Cada individuo tiene un área proporcional a su fitness.



Problemas:

- **Mar de mediocres**: si tengo 1 individuo con mucho fitness y muchos individuos con bajo fitness, es más probable que salga elegido uno de estos últimos.
- **Mar de virtuosos**:Si tengo fitness muy parecidos, va a elegir prácticamente al azar.
  Por lo tanto falla cuando la función de distribución (un histograma) es picuda.

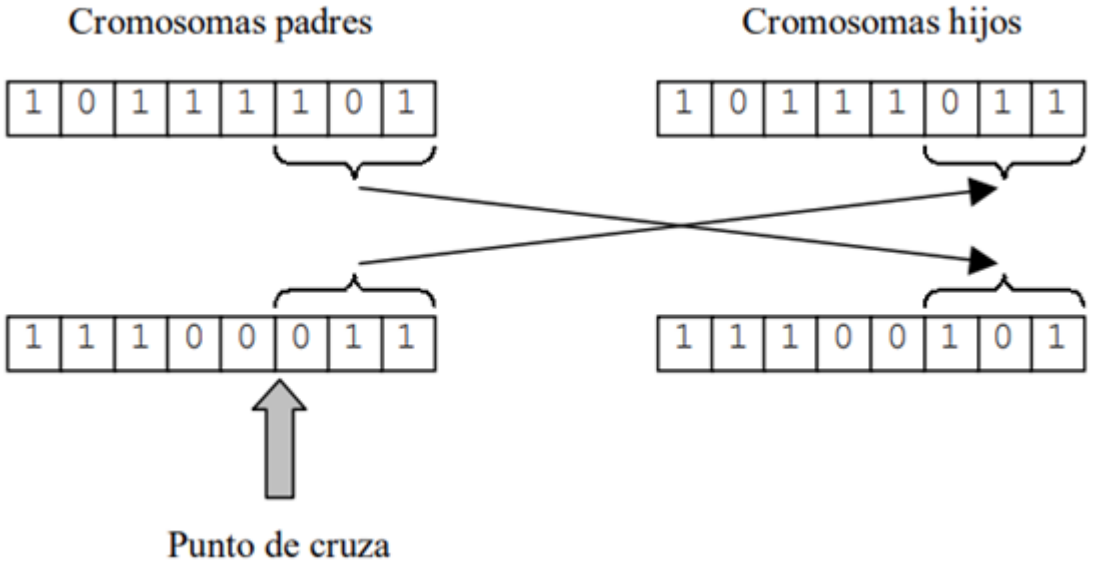
- **Ventanas**: En ventana, se ordenan de mayor a menor en fitness, y primero elige una ventana grande al azar, luego achica la ventana y elige al azar. De cada ventana elige un progenitor.



Competencias: Se eligen k individuos, y el que tiene mayor fitness gana

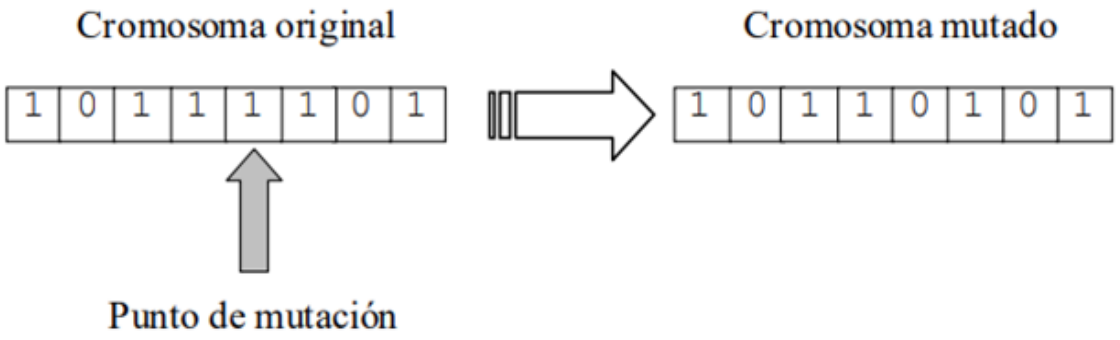
Operadores de variación

- **Cruzas simples**
  Se elige un punto al azar y se intercambian los genes.



El rol de esto es combinar las mejores partes de los padres, es decir es afinar la búsqueda del mínimo error en una superficie de error.

- **Mutaciones**
  Se toma un punto al azar de los hijos si se desea y se muta ese gen.



La tasa de mutación puede ser a nivel de individuos, a nivel de cromosomas o a nivel de genes.
En la búsqueda sirve para explorar regiones lejanas en el espacio.

- Mutación -> explora nuevos horizontes
- Cruzas simples -> refina la exploración

Demasiada mutación → casi aleatorio, no converge bien (pierde la capacidad de afinar).
Demasiada cruza (poca mutación) → converge rápido pero a un mínimo local, se pierde diversidad.

Operador de reemplazo durante la reproducción

- Reemplazo total: reemplazo la generación por todos los hijos
- Reemplazo con brecha generacional: hay generaciones en las que pueden convivir padres con hijos. Por ejemplo se eligen 10 padres y con esos 10 se generan los 90 restantes para llegar a el tamaño de la población de 100.
- Elitismo: se toma al mejor individuo de la población y se lo pasa a la siguiente generación. La consecuencia es que la función de fitness aumentará de a escalones.

Variantes de Computación Evolutiva (Estrategias de Evolución, Programación Genética)

Evolutivo / Estrategias de evolución

Nosotros anteriormente nos centramos en los genéticos nomas (cromosomas binarios). Pero ahora:

Si los parámetros tales como:

- Probabilidad de mutaciones
- Probabilidad de cruzas
- Tamaño de la población
- Brecha generacional
- Elitismo

Fueran adaptables, surgen los algoritmos llamados estrategias de evolución.

Para esto se utilizará:

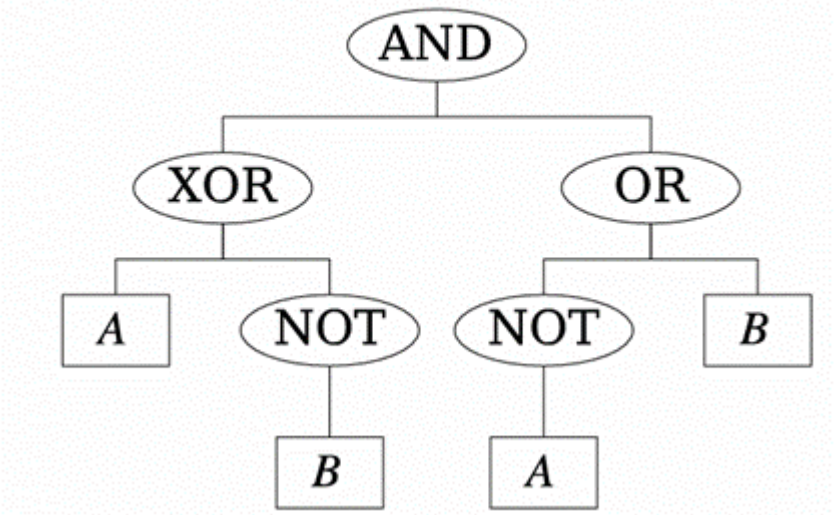
- Representación fenotípica
  Es decir que se pone el valor directamente en los cromosomas, sin convertirlos a binarios. Por ej, se ponen los pesos tal cual de una red neuronal.
  Dentro de esta representación tendremos:
    - Variables objetivo: las que venimos evolucionando, pesos, coordenadas, etc.
    - Variables de control: controlan la evolución ejemplo probabilidad de mutación.
- Fitness: igual que en algoritmos genéticos.
- Operadores:
    - Mutación (solo si no empeora)
    - Cruza (optativa)
- Selección: aleatoria, combinada con mutación. Se muta un padre elegido al azar y si ese padre mejoro, se lo deja para la próxima generación, sino se elimina.
- Reproducción: determinística, hay dos esquemas
    - Mecanismo : padres producen hijos.
      Próxima generación: eliminando los peores de
    - Mecanismo : padres producen hijos.
      Próxima generación: subconjunto con los mejores de

Programación genética

Es otro algoritmo. Es simplemente generar programas de forma automática.
Ejemplo
De los elementos básicos de un programa:
Variables y constantes
Operadores aritméticos y lógicos
Funciones matemáticas
Condicionales
Bucles
Recursiones

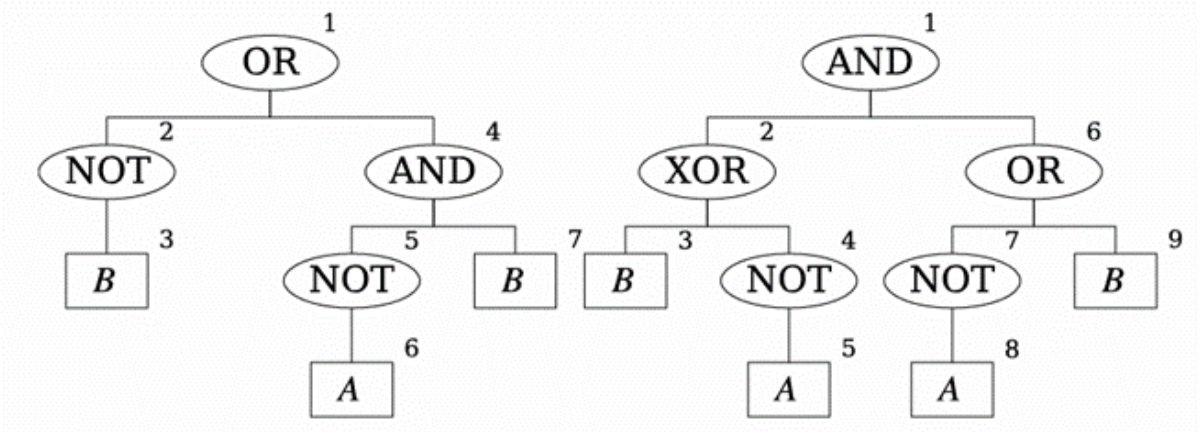
Podemos para hacer un ejemplo sencillo de operaciones lógicas y representarlo en un árbol:

$$((A)\text{XOR}(\text{NOT}(B)))\text{AND}((\text{NOT}(A))\text{OR}(B))$$

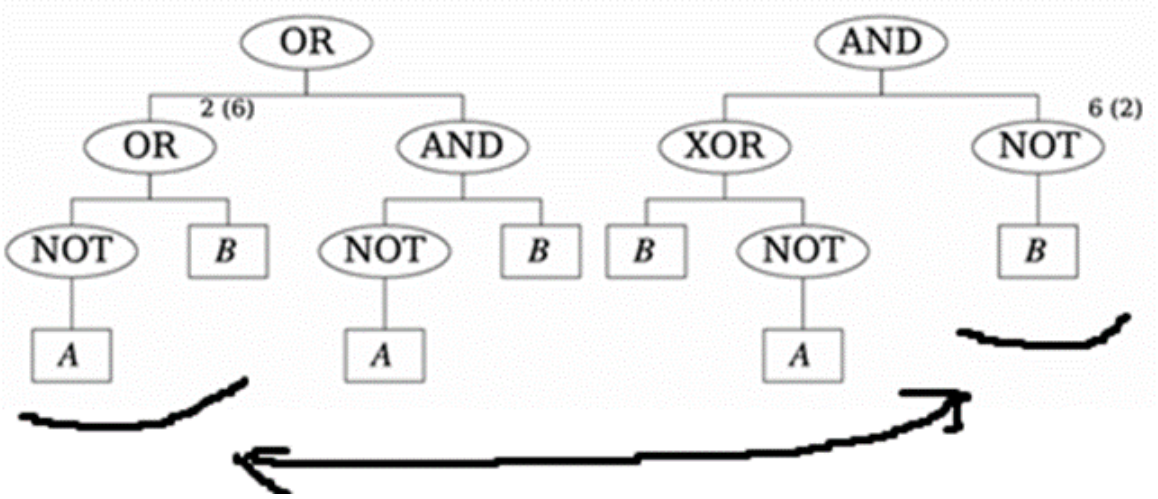


Cruzas en PG

1. Para los dos padres numero los nodos y elijo los puntos a cruzar, ej 2 y 6.



2. Hago la cruza intercambiando las ramas



Mutaciones en PG

1. Numeración de nodos
2. Selección de la rama a mutar
3. Generación de un árbol al azar
4. Mutación en base al reemplazo

UNIDAD 6

Colonia de Hormigas

sACO

1. **t=0 inicializamos feromonas con valores pequeños al azar** σ_{ij}
2. **Ubicar N hormigas en el nodo origen**
3. **Repetir**
   - **3.1. para cada hormiga k = 1,2,...N**
     - 3.1.1. $p^k(t) = \emptyset$ lista de nodos visitados vacío
     - **3.1.2. repetir**
       - 3.1.2.1. seleccionar el nodo según la probabilidad $p^k(t)$

$$p^k_{ij}(t) = \begin{cases} \dfrac{\sigma^\alpha_{ij}(t)}{\sum_{\forall u \varepsilon Ni} \sigma^\alpha_{iu}(t)} & si\ j \in Ni \\ 0 & \text{en otro caso} \end{cases}$$

ponemos u para no confundir con el j de arriba - Ni vecinos

       - 3.1.2.2. una vez calculada la probabilidad de cada nodo según la cant de feromona lo seleccionamos por ruleta o ventana
       - 3.1.2.3. agregamos el paso (i,j) al camino $p^k(t)$

       hasta alcanzar el destino
     - 3.1.3. eliminamos los ciclos en $p^k(t)$
     - 3.1.4. calculamos la longitud del camino encontrado $f(p^k(t))$
   - **3.2. Para cada conexión (i,j) TODOS LOS NODOS → evaporamos feromonas**

$$\sigma_{ij}(t) = (1 - \rho)\sigma_{ij}(t)$$

   - **3.3. Para cada conexión (i,j) DEL CAMINO RECORRIDO → depositamos feromonas**

$$\sigma_{ij}(t+1) = \sigma_{ij}(t) + \sum_{\forall k/(i,j)\in p^k(t)} 1/f(p^k(t))$$

     al camino más largo menos feromonas
     al camino mas corto mas feromonas
   - **3.4. t = t+1**

Hasta que todas las hormigas sigan el mismo camino

4. **Devolver el camino más corto**

AS

1. **t = 0 inicializamos feromonas al azar con valores pequeños** σ
2. **Ubicamos N hormigas en el origen**
3. **Repetir**
   - 3.1.1. para cada hormiga K = 1,2,...N
   - 3.1.2. $p^k(t) = \emptyset$ lista de nodos visitados vacío
   - **3.1.3. Repetir**
     - 3.1.3.1. Seleccionar el nodo según la probabilidad $p^k(t)$

$$p^k_{ij}(t) = \{\frac{\sigma^\alpha_{ij}(t)\eta^B_{ij}(t)}{\sum_{\forall u \varepsilon Ni} \sigma^\alpha_{iu}(t)\eta^B_{ij}(t)} \quad si\ j \in Ni^k$$

η_{ij} *es el deseo de moverse* d_{ij} *es la distancia que hay al sig nodo*

$\eta_{ij} = 1/d_{ij}$ si el camino es más corto mayor deseo, más largo menor deseo

N^k_i lista de nodos vecinos con tabu: lista sin nodos por los que la hormiga ya paso

- 3.1.3.2. una vez calculada la probabilidad de cada nodo según la cant de feromona lo seleccionamos por ruleta o ventana
- 3.1.3.3. agregamos el paso (i,j) al camino $p^k(t)$

Hasta llegar a destino

- 3.1.4. Calculamos la longitud del camino encontrado $f(p^k(t))$

3.2. Para cada conexión (i,j) TODOS LOS NODOS → evaporamos feromonas

$$\sigma_{ij}(t) = (1 - \rho)\sigma_{ij}(t)$$

3.3. Para cada conexión (i,j) DEL CAMINO RECORRIDO → depositamos feromonas

$$\Delta\sigma^k_{ij}(t) = \begin{cases} Q/f(p^k(t)) & Global \\ Q & Uniforme \\ Q/d_{ij} & Local \end{cases}$$

$$\sigma_{ij}(t+1) = \sigma_{ij}(t) + \sum_{\forall k/(i,j) \epsilon\, p^k(t)} \Delta\sigma^k_{ij}(t)$$

al camino más largo menos feromonas
al camino mas corto mas feromonas

3.4. t = t+1

Hasta que todas las hormigas sigan el mismo camino

4. Devolver el mejor camino

Enjambre de Partículas

Fórmulas clave:

posición $x_k(t+1) = x_k(t) + v_k(t+1)$

velocidad $v_{ki}(t+1) = v_{ki}(t) + c_1 r_{1i}([y_{ki} - x_{ki}(t)] + c_2 r_{2i}([\hat{y}_{ki} - x_{ki}(t)]$

gEP: $\hat{y}_i$ (sin k — es la misma para todas las partículas)

IEP: $\hat{y}_{ki}$ (con k — cada partícula tiene la suya)

gEP - Global Best - Topología estrella

1. Inicializamos las posiciones al azar
2. **Repetir**
    a. Para cada partícula k = 1,2,...N
        i. si $f(x_k(t)) < f(y_k) \rightarrow y_k = x_k(t)$ Ver si la posición actual es mejor que su posición individual
        ii. $f(y_k) < f(\hat{y}) \rightarrow \hat{y} = y_k$ Ver si la posición actual es mejor que la posición global

    b. Para cada partícula k=1,2,...N
    c. $v_{ki}(t+1) = v_{ki}(t) + c_1 r_{1i}([y_{ki} - x_{ki}(t)] + \rightarrow experiencia\ personal$
    $c_2 r_{2i}([\hat{y}_i - x_{ki}(t)]) \rightarrow experiencia\ social$

$[y_{ki} - x_{ki}(t)]$ → dist. entre la mejor pos que obtuvo el individuo y la pos actual

$[\hat{y}_i - x_{ki}(t)]$

    d. $x_k(t+1) = x_k(t) + v_k(t+1)$
    e. $t = t + 1$
    Hasta cumplir con condición de finalización
3. Devolver la mejor partícula encontrada

Inicializamos el azar

Repetimos

- Para cada partícula:
    - Si la posición actual en el instante t es mejor que la mejor posición individual la guardamos para luego comparar con la mejor pos global
    - Si la mejor posición actual de la partícula es mejor que la global, reasignamos la global
- Para cada partícula
    - Calculamos la velocidad del siguiente instante
    - Calculamos la posición del siguiente instante
    - C constante de aceleración - constante
    - R entre 0 y 1 aleatoria en cada it. sirve para que la bandada no caiga a un minimo local

lEP - Local best Topología anillo

1. Inicializamos las posiciones al azar
2. **Repetir**
   a. Para cada partícula k = 1,2,...N
      i. si $f(x_k(t)) < f(y_k) \rightarrow y_k = x_k(t)$
      ii. $f(y_k) < f(\hat{y}) \rightarrow \hat{y} = y_k$
   b. Para cada partícula k=1,2,...N
   c. $v_{ki}(t+1) = v_{ki}(t) + c_1 r_{1i}([y_{ki} - x_{ki}(t)] + c_2 r_{2i}([\hat{y}_{ki} - x_{ki}(t)]$ Aparece k ahora porque la información social depende de cada partícula k, no es la misma para todas

      $\hat{y}_k = argmin \forall y_u \epsilon \varepsilon_k \{f(y_u)\}$
   d. $x_k(t+1) = x_k(t) + v_k(t+1)$
   e. $t = t + 1$

   Hasta cumplir con condición de finalización
3. Devolver la mejor partícula encontrada

Inicializamos el azar
Repetimos

- Para cada partícula:
  - Si la posición actual en el instante t es mejor que la mejor posición individual, la guardamos
  - A diferencia de gEP, acá NO comparamos contra una posición global única — cada partícula k calcula su propio ŷ_k, buscando el mejor valor solo entre las partículas de su vecindad (ε_k), no de todo el enjambre
- Para cada partícula:
  - Calculamos la velocidad del siguiente instante (usando el ŷ_k propio de esa partícula, no uno compartido)
  - Calculamos la posición del siguiente instante
  - C constante de aceleración - R constante entre 0 y 1, aleatoria en cada iteración

DOS ALGORITMOS
gET Topología estrella: $\hat{y}$ es un solo valor global compartido por todo el enjambre
lEPt Topología anillo: $\hat{y}$ tiene subíndice k $\hat{y}_k$ cada partícula tiene su propia mejor referencia social

gBest: todas las partículas persiguen al mismo líder global.
lBest: cada partícula persigue al mejor de su propio vecindario — la buena información se propaga de a poco, vecino por vecino, en vez de llegar a todos de una.

UNIDAD 7

Teoría de Conjuntos Borrosos

Memorias Asociativas Borrosas (FAM)